



Scuola Arti e Mestieri Trevano

Sezione informatica

Implementazione di OraX in .NET MAUI



Titolo del progetto:

OraX

Alunno/a:

Linda Bytyqi, Tessa Caminada e Ryan Pinana

Classe:

I3AC

Anno scolastico:

2025/2026

Docente responsabile:

Ingrid Cereda



1 Sommario

2	Indice delle figure	4
3	Introduzione	5
3.1	Informazioni sul progetto	5
3.2	Scopo	5
4	Analisi	6
4.1	Analisi del dominio	6
4.2	User Stories	6
4.3	Analisi e specifica dei requisiti	9
4.3.1	Requisiti funzionali	9
4.3.2	Requisiti non funzionali	12
4.3.3	Spiegazione elementi tabella dei requisiti	12
4.3.4	Elementi della tabella	12
4.4	Use case	13
4.5	Pianificazione	14
4.5.1	Diagramma di Gantt	1
4.5.2	Periodo	1
4.5.3	Struttura	1
4.5.4	Dipendenze e controllo	2
4.5.5	Conclusione	2
4.6	Analisi dei mezzi	2
4.6.1	Software	3
4.6.2	Hardware	3
5	Progettazione	4
5.1	Sprint	4
5.1.1	Primo Sprint	4
5.1.2	Secondo Sprint	4
5.1.3	Terzo Sprint	5
5.2	Design dell'architettura del sistema	5
5.3	Design dei dati e database	6
5.4	Design delle interfacce	7
5.4.1	Pagina di login	7
5.4.2	Pagina di registrazione	8
5.4.3	Pagina home	8
5.4.4	Pagina dettagli attività	9
5.4.5	Notifiche	10
5.4.6	Pagina del profilo	11
5.4.7	Pagina modifiche	12
5.4.8	Pagina Statistiche	13
5.4.9	Pagina condivisioni	14
5.4.10	Notifica condivisione	15
5.5	Design procedurale	16
5.5.1	Flusso generale dell'applicazione	16
5.5.2	Gestione dell'autenticazione	18
5.5.3	Gestione delle attività	18
5.5.4	Sistema di notifiche	18
5.5.5	Sistema di ricerca	19
5.5.6	Gestione delle statistiche	19
5.5.7	Gestione della condivisione	19
5.5.8	Struttura logica delle componenti	19
5.5.9	Gestione degli errori	19
5.5.10	Considerazioni finali	20
6	Implementazione	21
6.1	Creazione dell'applicativo	21
6.2	Struttura generale	21



6.3	Database SQLite	22
6.4	Pagina di login	22
6.4.1	User.cs	22
6.4.2	Sicurezza – PasswordHelper.cs	23
6.4.3	Database – DatabaseService.cs	23
6.4.4	LoginPage.xaml	24
6.5	Pagina di registrazione	25
6.5.1	Toggle – visibilità password	26
6.5.2	Logica	27
6.6	Calendario	28
6.6.1	Struttura	28
6.6.2	Grafica	29
6.6.3	Collegamento tra interfaccia e codice C#	30
6.6.4	Data selezionata	30
6.6.5	Collezioni usate per mostrare i dati	31
6.6.6	Caricamento pagina iniziale	31
6.6.7	Gestione tipi di attività	31
6.6.8	Creazione e caricamento dei calendari	32
6.6.9	Memorizzazione dell'ultimo calendario selezionato	33
6.6.10	Caricamento delle attività del calendario	33
6.6.11	Inserimento degli eventi nel calendario grafico	34
6.6.12	Visualizzazione degli eventi del giorno	35
6.6.13	Apertura dell'overlay per aggiungere un evento	36
6.6.14	Aggiunta di una nuova attività	37
6.6.15	Eliminazione attività	40
6.6.16	Completamento attività	40
6.6.17	Ricerca delle attività	41
6.6.18	Visualizzazione del dettaglio di un'attività	42
6.6.19	Creazione di un nuovo calendario	42
6.6.20	Cambio del calendario selezionato	43
6.6.21	Condivisione del calendario con altri utenti	44
6.6.22	Gestione richieste ricevute	44
6.6.23	Database e salvataggio dei dati	45
6.6.24	Gestione del tema grafico	45
6.7	Statistiche, Profilo, Modifica Profilo e Logout	45
6.7.1	Statistiche	46
6.7.2	Profilo	47
6.7.3	Modifica profilo	49
6.7.4	Disconnessione	51
6.8	Notifiche	52
6.9	Conclusione	53
7	Test	54
7.1	Protocollo di test	54
7.2	Risultati test	60
7.3	Mancanze/limitazioni conosciute	60
8	Consuntivo	61
8.1	Differenze – Gantt Preventivo e Consuntivo	62
9	Conclusioni	63
9.1	Sviluppi futuri	64
9.2	Considerazioni personali	64
9.2.1	Linda	64
9.2.2	Tessa	65
9.2.3	Ryan	65
10	Glossario	66
11	Bibliografia	67
12	Allegati	67



2 Indice delle figure

Figura 1 - Use Case.....	14
Figura 2 - Gantt Preventivo	15
Figura 3 - Diagramma ER.....	6
Figura 4 - Diagramma ER.....	6
Figura 5 - Pagina Login Mockup.....	7
Figura 6 - Pagina Registrazione	8
Figura 7 - Pagina Registrazione Mockup	8
Figura 8 - Pagina Calendario Mockup	9
Figura 9 - Pagina Attività Mockup.....	10
Figura 10 - Notifiche Mockup.....	10
Figura 11 - Pagina Profilo Mockup	11
Figura 12 - Pagina Modifiche Mockup	12
Figura 13 - Pagina Statistiche Mockup.....	14
Figura 14 - Pagina Condivisioni Mockup	15
Figura 15 - Notifica Condivisione Mockup	15
Figura 16 - Diagramma di flusso	17
Figura 17 - Gantt Consuntivo	61



3 Introduzione

3.1 Informazioni sul progetto

- **Titolo:** OraX;
- **Allievi:** Linda Bytyqi, Ryan Pinana e Tessa Caminada;
- **Sede:** CPT Trevano;
- **Docente:** Ingrid Cereda;
- **Modulo:** M306 – SAMT Progetti;
- **Periodo:** 16.01.2026 – 29.05.2026

3.2 Scopo

Lo scopo del progetto OraX è quello di realizzare un'applicazione digitale per la gestione del tempo e delle attività personali, progettata per aiutare gli utenti ad organizzare in modo semplice, efficiente e intuitivo i propri impegni quotidiani.

L'app è pensata per studenti, lavoratori e utenti comuni che desiderano pianificare le proprie attività, monitorare le scadenze e migliorare la propria produttività, senza dover utilizzare strumenti complessi o poco accessibili.

Attraverso un'interfaccia chiara e personalizzabile, OraX consente di creare, modificare e organizzare attività all'interno di un calendario personale, offrendo funzionalità avanzate come notifiche automatiche, ricerca delle attività, statistiche sull'utilizzo e la possibilità di condividere calendari con altri utenti.

Il progetto mira ad unire organizzazione, tecnologia e usabilità, offrendo un prodotto che:

- migliori la gestione del tempo e delle priorità;
- renda la pianificazione delle attività più semplice e intuitiva;
- favorisca la collaborazione attraverso calendari condivisi;
- dimostri come la tecnologia possa supportare la produttività personale.

Inoltre, OraX intende mostrare come sia possibile sviluppare un'applicazione completa e funzionale, combinando sicurezza, personalizzazione e strumenti di analisi in un unico sistema integrato.

In sintesi, l'applicativo ha come obiettivo principale quello di offrire una soluzione moderna e accessibile per l'organizzazione delle attività, adatto ad un pubblico ampio.



4 Analisi

4.1 Analisi del dominio

OraX nasce per offrire una soluzione semplice e moderna per la gestione del calendario e delle attività personali. L'applicazione è pensata per essere utilizzata in contesti personali, scolastici e lavorativi, su dispositivi digitali. Supporta sia l'uso individuale sia la collaborazione tra utenti, ad esempio attraverso calendari condivisi e punta a garantire un'esperienza fluida anche in situazioni di connessione limitata.

Gli utenti principali di OraX sono studenti, lavoratori e persone che desiderano organizzare meglio il proprio tempo. I loro bisogni includono la possibilità di pianificare attività, ricevere notifiche per le scadenze, personalizzare il calendario e tenere traccia dei propri impegni tramite statistiche. In generale, si presuppone che gli utenti abbiano competenze digitali di base, quindi l'app deve essere facile da usare, intuitiva e accessibile anche per chi non ha molta esperienza con la tecnologia.

Esistono già applicazioni simili, come Google Calendar, ma spesso risultano troppo complesse o non completamente orientate alla personalizzazione e all'analisi delle attività personali. OraX viene proposta come un'alternativa più semplice e mirata, che unisce le funzioni essenziali di un calendario con strumenti utili per migliorare l'organizzazione e la produttività.

Il progetto si basa su convenzioni comuni nel settore delle app di gestione del tempo, come l'uso di viste giornaliere, settimanali e mensili, colori e icone per distinguere le attività e notifiche automatiche per ricordare le scadenze. In sintesi, OraX vuole essere uno strumento pratico, accessibile e utile per aiutare le persone a gestire meglio il proprio tempo nella vita di tutti i giorni.

4.2 User Stories

Le User Stories descrivono le funzionalità dell'applicazione dal punto di vista dell'utente. Servono a definire in modo semplice e chiaro cosa deve fare il sistema e quale valore offre all'utente finale. Ogni user story rappresenta un bisogno o un obiettivo che l'utente vuole raggiungere utilizzando l'applicazione. In questo modo è possibile sviluppare il progetto mantenendo sempre l'attenzione sull'esperienza dell'utente e sulle funzionalità più importanti.

ID: US-01		Story points: 3
Nome	Come nuovo utente voglio potermi registrare per vedere il mio calendario ed organizzare le mie attività.	
Stima	Media	
Criteri di accettazione		
001	La registrazione crea un account che andrà salvato nel database.	
002	Per fare l'accesso al nuovo account si dovranno utilizzare le proprie credenziali.	
003	L'utente non può accedere all'account di un amministratore.	
004	La password deve avere minimo 8 caratteri, compresi quelli speciali.	
005	L'utente deve riuscire a vedere il suo calendario.	
ID: US-02		Story points: 5
Nome	Come utente voglio inserire una nuova attività e identificarne i dettagli per salvare i miei appuntamenti.	



Stima	Grande
Criteri di accettazione	
001	L'utente premendo sul giorno visualizzerà il form per creare una nuova attività
002	Le attività verranno salvate nel profilo.
003	Visualizzazione delle attività nel calendario.
ID: US-03	
Story points: 5	
Nome	Come utente autenticato voglio modificare ed eliminare le attività create per cambiare le proprie informazioni
Stima	Media
Criteri di accettazione	
001	Dopo la modifica il database viene aggiornato.
002	Cliccando la X si potranno eliminare le attività.
003	Cliccando "Edit" si potranno modificare le attività e i suoi dettagli.
004	Visivamente le attività eliminate dovranno scomparire.
ID: US-04	
Story points: 3	
Nome	Come utente voglio personalizzare tema e colori del mio calendario così da adattarlo alle mie preferenze
Stima	Piccola
Criteri di accettazione	
001	I temi e i colori scelti dall'utente dovranno rimanere tali a tutti gli accessi di quell'utente.
002	Cliccando sul profilo si potranno modificare temi e colori.
003	Saranno disponibili tutte le shade di colori.
ID: US-05	
Story points: 8	
Nome	Come utente voglio ricevere una notifica alla scadenza di un'attività così da non dimenticare i miei impegni
Stima	Grande
Criteri di accettazione	
001	Le notifiche dovranno essere popup.
002	L'arrivo delle notifiche dovrà essere accompagnato da un suono.
003	L'utente dovrà scegliere nelle impostazioni dell'attività quanto prima ricevere la notifica di avviso.
004	L'utente può disattivare le notifiche.
ID: US-06	
Story points: 13	



Nome	Come utente registrato voglio visualizzare statistiche sulle mie attività così da analizzare la mia produttività e organizzazione	
Stima	Grande	
Criteri di accettazione		
001	Le statistiche saranno salvate nel database.	
002	Le statistiche saranno visualizzate dall'utente nella pagina apposita.	
003	Le statistiche si aggiorneranno automaticamente ad ogni modifica dell'attività.	
ID: US-07		Story points: 5
Nome	Come utente registrato voglio cercare le mie attività così da trovare rapidamente un impegno specifico	
Stima	Piccolo	
Criteri di accettazione		
001	La barra di ricerca sarà situata in cima alla pagina principale.	
002	Quando l'utente invia la ricerca i risultati saranno visualizzati al posto del calendario.	
003	Dovrà essere applicare alla ricerca dei filtri.	
004	Cliccando sul risultato interessato l'utente sarà reindirizzato all'attività nella pagina del calendario.	
ID: US-08		Story points: 13
Nome	Come utente registrato voglio creare un calendario condiviso con un altro utente registrato così da gestire attività comuni	
Stima	Grande	
Criteri di accettazione		
001	Il calendario condiviso funzionerà anche in assenza di connessione (modalità offline).	
002	Si potranno condividere il calendario max. con 3 persone.	
003	Le attività potranno essere create, modificate ed eliminate da tutti gli utenti del calendario.	
ID: US-09		Story points: 13
Nome	Come utente registrato voglio inviare una richiesta ad un altro utente per condividere un calendario così da collaborare nella pianificazione di attività comuni.	
Stima	Grande	
Criteri di accettazione		
001	L'utente ospite (quello che non crea il calendario) riceverà una notifica per accettare la richiesta di condivisione.	
002	Le attività condivise saranno visualizzabili solo nel calendario condiviso	
ID: US-010		Story points: 1



Nome	Come utente registrato voglio potermi disconnettere dall'account per uscire dall'app
Stima	Piccolo
Criteri di accettazione	
001	Il tasto di logout sarà nel menu del profilo.
002	Ci sarà un'avvertenza popup per chiedere all'utente se è sicuro di voler uscire.
003	Dopo il logout se si volesse rientrare sarà obbligatorio fare nuovamente il login.

4.3 Analisi e specifica dei requisiti

In questo capitolo vengono descritte le funzionalità che il sistema dovrà offrire e le caratteristiche qualitative richieste, in modo che lo sviluppo possa basarsi su una base chiara e condivisa.

4.3.1 Requisiti funzionali

ID: REQ-01	
Nome	Registrazione al calendario
Priorità	1
Versione	1.0
Note	Un nuovo utente deve poter registrarsi all'applicativo inserendo le sue informazioni private.
Sotto requisiti	
001	Si necessita di una banca dati per l'archiviazione dei profili.
002	Le password devono avere minimo otto caratteri e devono contenere caratteri speciali.
ID: REQ-02	
Nome	Accesso al proprio profilo
Priorità	1
Versione	1.0
Note	L'utente deve poter accedere al suo profilo utilizzando le sue credenziali.
Sotto requisiti	
001	Si necessita di una banca dati per l'archiviazione dei profili.
002	Si necessita il corretto funzionamento del REQ-01.
ID: REQ-03	



Nome	Visualizzazione del proprio calendario
Priorità	1
Versione	1.0
Note	Una volta eseguito l'accesso, l'utente potrà vedere il suo calendario.
Sotto requisiti	
001	Si necessita il corretto funzionamento del REQ-02.
ID: REQ-04	
Nome	Inserimento di un'attività
Priorità	1
Versione	1.0
Note	L'utente nel suo calendario potrà inserire un'attività.
Sotto requisiti	
001	Si necessita il corretto funzionamento del REQ-03.
ID: REQ-05	
Nome	Informazioni sulle attività
Priorità	1
Versione	1.0
Note	L'attività inserita avrà delle informazioni opzionali da specificare.
Sotto requisiti	
001	Si necessita il corretto funzionamento del REQ-04.
002	Ogni categoria avrà un'icona per identificarla (scelta dall'utente).
003	Le attività potranno essere rincorrenti o dipendenti una dall'altra.
ID: REQ-06	
Nome	Eliminazione e modifica di un'attività
Priorità	1
Versione	1.0
Note	L'utente ha la possibilità di eliminare o modificare le attività.
Sotto requisiti	
001	Si necessita il corretto funzionamento del REQ-04.
ID: REQ-07	



Nome	Personalizzazione del proprio calendario
Priorità	2
Versione	1.0
Note	Si potrà scegliere il tema e i colori del proprio calendario.
Sotto requisiti	
001	Si necessita il corretto funzionamento del REQ-03.
ID: REQ-08	
Nome	Notifiche
Priorità	1
Versione	1.0
Note	L'utente riceverà una notifica il giorno prima della scadenza dell'attività.
Sotto requisiti	
001	Si necessita il corretto funzionamento del REQ-04.
ID: REQ-09	
Nome	Statistiche
Priorità	1
Versione	1.0
Note	Nella pagina dedicata al profilo dell'utente, si potranno vedere le varie statistiche che comprendono l'analisi delle attività.
Sotto requisiti	
001	Avere un account.
ID: REQ-10	
Nome	Ricerca delle attività
Priorità	1
Versione	1.0
Note	L'utente potrà ricercare le proprie attività.
Sotto requisiti	
001	Si necessita il corretto funzionamento del REQ-04.
ID: REQ-11	
Nome	Calendario condiviso



Priorità	1
Versione	1.0
Note	L'utente può creare un calendario condiviso con un altro utente dell'applicativo.
Sotto requisiti	
001	Il calendario condiviso deve funzionare anche in assenza di rete.
ID: REQ-12	
Nome	Riunioni
Priorità	1
Versione	1.0
Note	Si potrà mandare la richiesta ad un altro utente per il calendario condiviso.
Sotto requisiti	
001	Il calendario condiviso deve funzionare anche in assenza di rete.

4.3.2 Requisiti non funzionali

I requisiti non funzionali riguardano le caratteristiche di qualità, i vincoli e le condizioni generali che il sistema deve rispettare.

- **Sicurezza:** questo requisito è fondamentale per un applicativo che gestisce attività private. I dati degli utenti verranno archiviati in modo sicuro e le credenziali dovranno rispettare criteri di password complesse. Inoltre, sarà previsto un logout automatico nel momento in cui l'utente chiude l'applicazione;
- **Prestazioni:** il calendario dovrà aggiornarsi in modo rapido ed efficiente in caso di inserimento, modifica o eliminazione di un'attività. Le notifiche dovranno essere inviate in modo puntuale e affidabile;
- **Usabilità:** l'interfaccia dovrà essere intuitiva, chiara e semplice da utilizzare, anche per utenti meno esperti;
- **Scalabilità:** il sistema dovrà supportare l'aggiunta di nuovi utenti e la crescita della banca dati senza compromettere le prestazioni;
- **Integrità dei dati:** le informazioni dovranno rimanere coerenti, corrette e protette, e tutte le modifiche dovranno essere salvate in modo affidabile.

4.3.3 Spiegazione elementi tabella dei requisiti

La tabella dei requisiti funzionali rappresenta un riepilogo strutturato di tutte le caratteristiche che il sistema deve possedere. Ogni requisito è identificato da un codice univoco e descritto attraverso una serie di campi che ne specificano la natura, la priorità, la versione e le eventuali relazioni con altri requisiti.

4.3.4 Elementi della tabella

ID:



È l'identificativo univoco del requisito.

Serve per distinguere in modo chiaro ogni requisito dagli altri e per poterlo facilmente richiamare all'interno della documentazione.

Generalmente viene espresso nella forma REQ-XX (es. REQ-01 per il primo requisito).

Nome:

Indica una breve descrizione del requisito, ossia il suo titolo sintetico che riassume in poche parole la funzionalità o la caratteristica richiesta.

Ad esempio: "Visualizzazione del calendario".

Priorità:

Specifica l'importanza del requisito all'interno del progetto.

La priorità viene definita in accordo con il committente e può avere diversi livelli (ad esempio:

- 1 = alta → requisito fondamentale, necessario per il funzionamento del sistema;
- 2 = media → requisito importante ma non essenziale;
- 3 = bassa → requisito secondario o opzionale).

La priorità aiuta a stabilire quali requisiti devono essere implementati per primi.

Versione:

Indica la versione del requisito, utile per tenere traccia di eventuali modifiche nel tempo.

Ogni aggiornamento o revisione comporta un incremento della versione (es. 1.0 → 1.1 → 2.0).

Nella documentazione principale viene mostrata solo l'ultima versione, mentre le precedenti vengono archiviate nei diari di progetto.

Note:

Campo dedicato a eventuali osservazioni aggiuntive, specifiche tecniche, riferimenti a fonti esterne o collegamenti con altri requisiti.

Ad esempio: "Si necessita il corretto funzionamento del REQ-01" oppure "Inserimento delle attività".

Sotto-requisiti:

Elenca gli elementi più specifici o le sotto-funzionalità che compongono il requisito principale.

Servono per descrivere in modo più dettagliato come il requisito sarà realizzato o quali condizioni devono essere soddisfatte affinché il requisito sia considerato completo.

Ad esempio:

- REQ-14 (Eliminazione dell'account) può avere come sotto-requisiti:
- REQ-14.001: "Si necessita il corretto funzionamento del REQ-02";
- REQ-14.002: "L'utente deve avere un account"

4.4 Use case

Un Use Case (caso d'uso) descrive un insieme di azioni o interazioni tra un attore e un sistema. Rappresenta il comportamento funzionale del sistema dal punto di vista dell'utente, definendo cosa il sistema deve fare, senza entrare nei dettagli di come lo farà.

Nel contesto del progetto OraX, l'attore principale è:

- **Utente** → colui che utilizza l'app per organizzare le sue attività quotidiane.

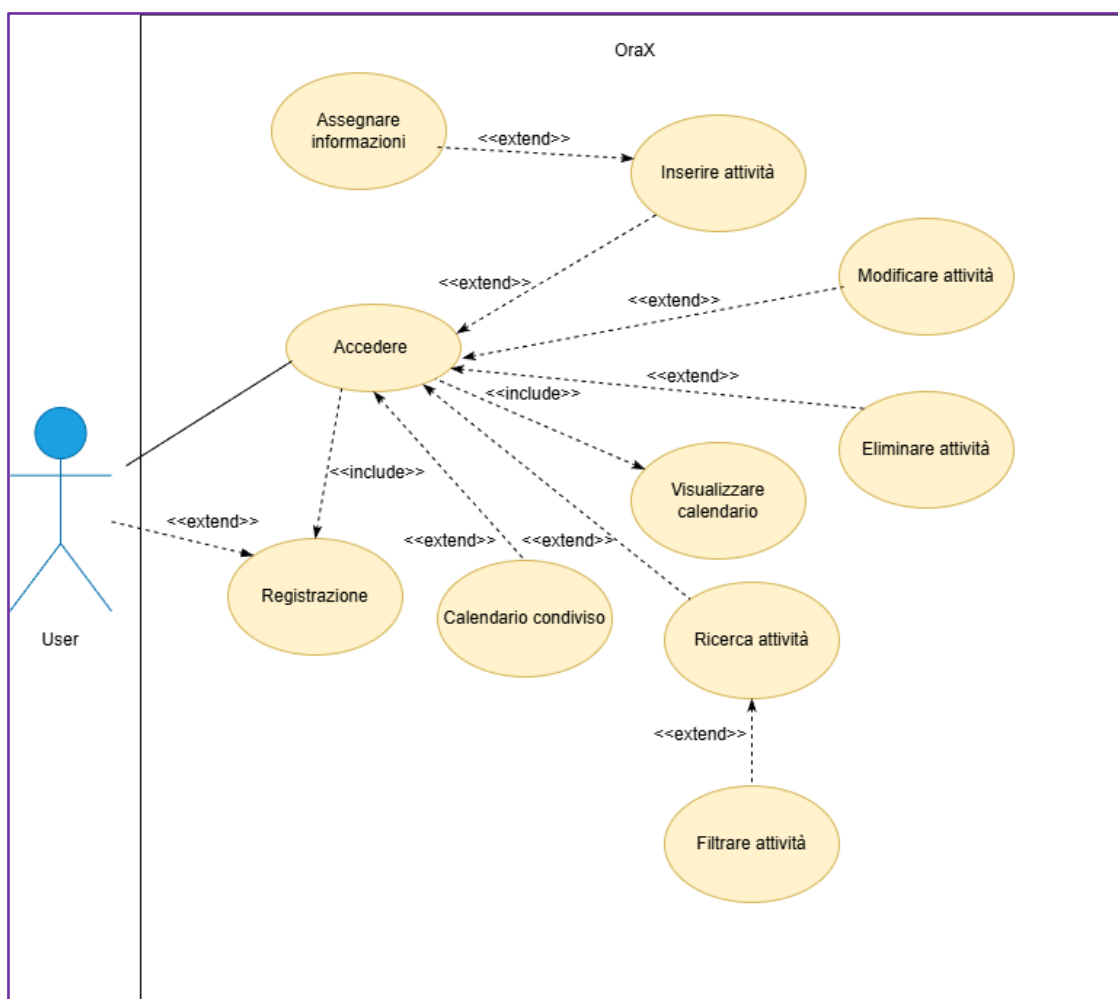


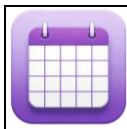
Figura 1 - Use Case

4.5 Pianificazione

La pianificazione rappresenta una fase fondamentale nello sviluppo del progetto, in quanto consente di organizzare le attività, definire le priorità e distribuire il lavoro tra i membri del team in modo strutturato ed efficiente.

Per il progetto OraX è stato utilizzato un diagramma di Gantt, strumento che permette di rappresentare graficamente le attività previste, la loro durata e la suddivisione temporale del lavoro durante l'intero periodo di sviluppo.

L'utilizzo del diagramma di Gantt ha consentito di monitorare l'avanzamento del progetto, verificare il rispetto delle scadenze e coordinare in maniera più efficace le diverse fasi di implementazione.



SAMT – Sezione Informatica

Implementazione di OraX in .NET MAUI

Pagina 15 di 82

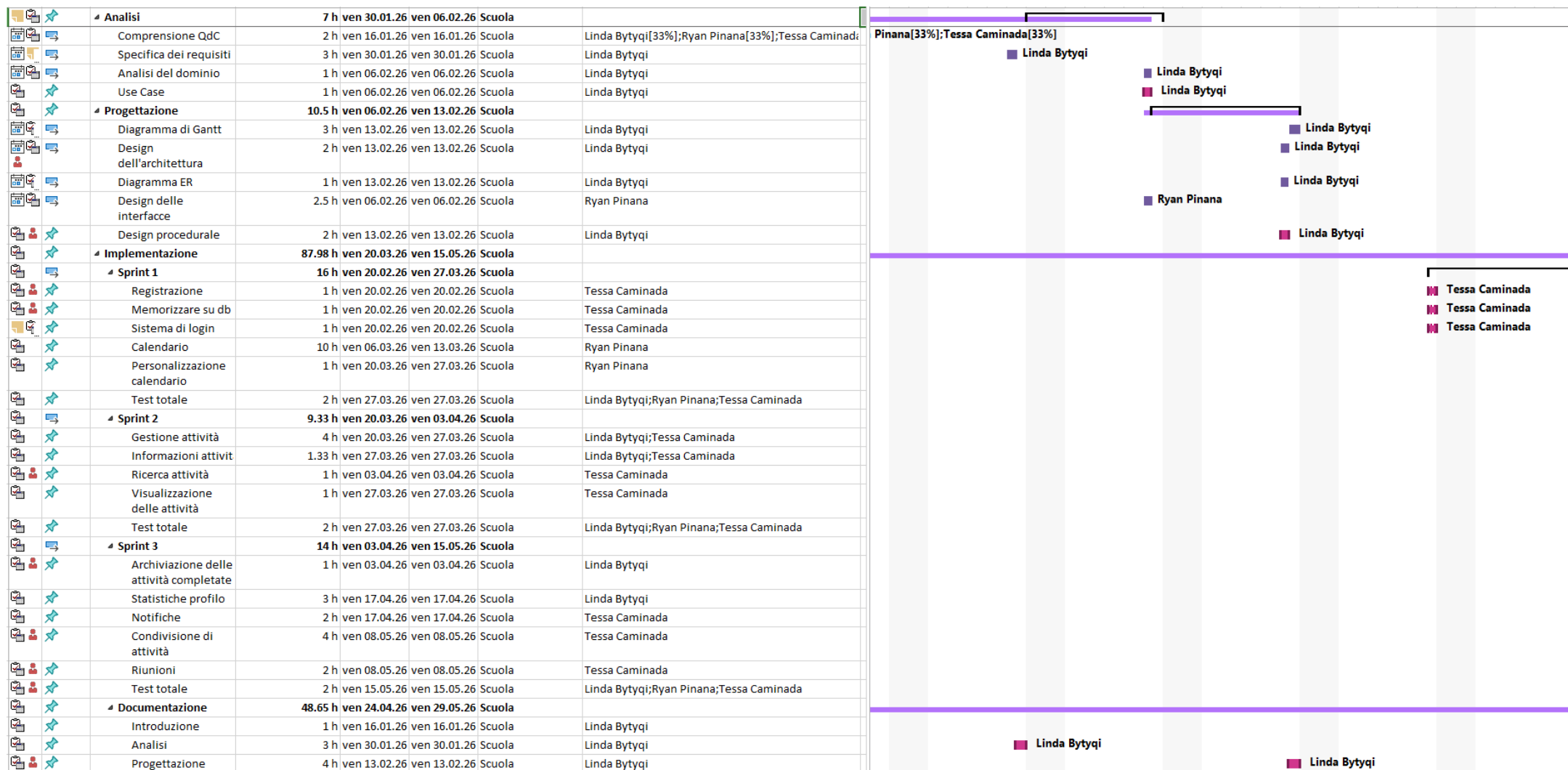


Figura 2 - Gantt Preventivo



4.5.1 Diagramma di Gantt

Il diagramma di Gantt rappresenta la pianificazione temporale delle attività del progetto OraX, evidenziando le principali fasi di sviluppo, la distribuzione del lavoro e le relazioni di dipendenza tra le diverse attività.

Attraverso questo strumento è stato possibile organizzare il progetto in maniera progressiva e controllata, garantendo una migliore gestione delle tempistiche e delle risorse disponibili.

4.5.2 Periodo

Il progetto si sviluppa nel periodo compreso tra il 16.01.2026 e il 29.05.2026, come definito nella pianificazione generale del modulo M306.

Il diagramma consente di monitorare l'evoluzione del progetto nel tempo, verificando il rispetto delle scadenze previste e individuando eventuali sovrapposizioni o ritardi nelle attività pianificate.

4.5.3 Struttura

Il lavoro è stato suddiviso in quattro fasi principali:

1. **Analisi:** comprende lo studio del dominio applicativo e la definizione delle funzionalità richieste dal sistema. Le principali attività svolte sono:
 - analisi del dominio;
 - analisi dei requisiti funzionali e non funzionali;
 - definizione delle User Stories;
 - realizzazione degli Use Case;
 - raccolta delle informazioni necessarie allo sviluppo.

Questa fase ha l'obiettivo di definire una base progettuale chiara e completa, necessaria per le successive attività di progettazione e implementazione.

2. **Progettazione:** è dedicata alla definizione dell'architettura del sistema e dell'interfaccia grafica dell'applicazione. Comprende:
 - realizzazione del diagramma di Gantt;
 - progettazione del database;
 - definizione dell'architettura del sistema;
 - creazione dei mockup delle interfacce;
 - definizione del design procedurale.

Questa fase consente di organizzare in modo strutturato le componenti del sistema prima dell'effettiva implementazione.

3. **Implementazione:** rappresenta la parte principale del progetto e comprende lo sviluppo delle funzionalità dell'applicazione tramite .NET MAUI e SQLite. Le principali funzionalità sviluppate sono:
 - sistema di registrazione e login;
 - visualizzazione del calendario;
 - gestione delle attività;
 - modifica ed eliminazione delle attività;
 - sistema di notifiche;
 - ricerca delle attività;
 - statistiche;
 - calendario condiviso;
 - gestione del profilo utente.

Titolo del progetto: OraX
Alunno/a: Linda Bytyqi, Tessa Caminada e Ryan Pinana
Classe: I3AC
Anno scolastico: 2025/2026
Docente responsabile: Ingrid Cereda



Le attività vengono sviluppate in maniera progressiva, seguendo l'organizzazione definita dagli Sprint e rispettando le dipendenze funzionali tra le varie componenti del sistema.

4. Documentazione: viene sviluppata parallelamente alle altre fasi del progetto, così da mantenere costantemente aggiornati:
 - analisi;
 - progettazione;
 - implementazione;
 - test;
 - consuntivi finali.

Questa scelta consente di garantire maggiore coerenza tra il lavoro svolto e la documentazione finale del progetto.

4.5.4 Dipendenze e controllo

Nel diagramma di Gantt sono presenti le dipendenze tra le varie attività, rappresentate tramite collegamenti che indicano l'ordine di esecuzione e i vincoli temporali tra le diverse fasi del progetto.

Il diagramma permette inoltre di confrontare la pianificazione iniziale con lo stato di avanzamento reale del progetto, facilitando il monitoraggio delle attività e l'individuazione di eventuali ritardi.

4.5.5 Conclusione

Il diagramma di Gantt fornisce una visione chiara e strutturata dell'intero progetto OraX, consentendo una gestione più efficace delle attività, del tempo e delle risorse disponibili.

L'utilizzo di questo strumento ha contribuito a migliorare l'organizzazione del lavoro del team, favorendo il rispetto delle scadenze e una pianificazione più precisa delle diverse fasi di sviluppo.

4.6 Analisi dei mezzi

Per lo sviluppo del progetto **OraX** verrà adottata la piattaforma **.NET MAUI (Multi-platform App UI)**, un framework sviluppato da Microsoft che consente la creazione di applicazioni multipiattaforma attraverso un'unica base di codice in **C#** e **XAML**. Tale tecnologia permette di realizzare applicazioni eseguibili su diversi sistemi operativi, tra cui Windows, Android e iOS, contribuendo a ridurre i tempi di sviluppo e a semplificare le attività di manutenzione.

L'utilizzo di .NET MAUI risulta particolarmente adeguato per questo progetto, in quanto offre strumenti avanzati per la realizzazione di interfacce grafiche moderne, fluide e altamente personalizzabili, caratteristiche fondamentali per garantire un'esperienza utente efficace e intuitiva. Inoltre, il framework mette a disposizione funzionalità integrate per la gestione della navigazione, dei dati, delle notifiche e dell'interazione con il database, elementi essenziali per l'implementazione delle funzionalità previste dal sistema.

Dal punto di vista operativo, lo sviluppo verrà effettuato mediante l'ambiente **Visual Studio 2022**, che offre strumenti avanzati per la scrittura del codice, il debugging e il testing dell'applicazione, assicurando elevati standard qualitativi durante tutte le fasi di sviluppo.

Per quanto riguarda la persistenza dei dati, verrà utilizzato un database locale, in grado di garantire la memorizzazione sicura e duratura delle informazioni relative agli utenti e alle attività. In particolare, si farà uso di **SQLite**, un database leggero che consente di salvare i dati direttamente sul dispositivo dell'utente.

SQLite rappresenta una soluzione ottimale in quanto permette di mantenere i dati anche in assenza di connessione internet e offre un'elevata efficienza nelle operazioni di lettura e scrittura. L'integrazione con .NET MAUI avverrà tramite la libreria **SQLite-net-pcl**, che consente di gestire il database in modo semplice ed efficace utilizzando il linguaggio C#.



Questa scelta tecnologica garantisce buone prestazioni, facilità di sviluppo e un'elevata scalabilità, rendendo possibile l'estensione futura del sistema senza la necessità di interventi strutturali significativi.

Per ulteriori dettagli e approfondimenti, si consiglia di consultare la documentazione ufficiale di Microsoft Learn e di SQLite.

4.6.1 Software

Per lo sviluppo del progetto è stato utilizzato l'ambiente di sviluppo integrato Microsoft Visual Studio 2022, uno strumento gratuito e completo per la realizzazione di applicazioni destinate a diversi dispositivi, tra cui computer, tablet e smartphone, oltre che per lo sviluppo di siti web e servizi.

Di seguito sono riportate alcune informazioni relative all'ambiente di sviluppo utilizzato:

- **Versione:** 1.78.2 (System Setup)
- **Data di rilascio:** 10/05/2023
- **Electron:** 22.5.2
- **Sistema operativo:** Windows_NT x64 10.0.22631

Per la realizzazione del progetto sono stati inoltre impiegati i seguenti strumenti software:

- Microsoft Word 16.0 – per la redazione della documentazione
- Microsoft Project 16.0 – per la pianificazione e gestione del progetto
- XAMPP Control Panel v3.3.0 – per la gestione dell'ambiente server locale
- MySQL 8.0 – per la gestione dei database
- Microsoft Visual Studio 2022 – per lo sviluppo dell'applicazione

L'insieme di questi strumenti ha consentito di coprire in modo completo tutte le fasi del progetto, dalla progettazione alla realizzazione, fino alla documentazione finale.

4.6.2 Hardware

Il prodotto sviluppato è progettato per essere eseguito su diverse piattaforme e dispositivi, tra cui computer, smartphone e tablet, purché dotati del software necessario per l'esecuzione dell'applicazione.

Per la realizzazione e il testing del progetto è stato utilizzato un computer scolastico con sistema operativo Windows 11 Education installato.

Di seguito sono riportate le specifiche hardware del dispositivo utilizzato:

- **Processore:** Intel Core i7-13700 – frequenza base 2.10 GHz
- **Scheda grafica:** NVIDIA T400
- **Memoria RAM:** 32 GB DDR4

La configurazione hardware utilizzata garantisce prestazioni elevate durante le fasi di sviluppo, esecuzione e testing dell'applicazione, permettendo una gestione fluida delle risorse e degli strumenti impiegati.



5 Progettazione

Per la gestione dei dati è stato scelto di utilizzare il database SQLite, integrato all'interno dell'applicazione tramite una libreria (pacchetto NuGet) che consente l'utilizzo di SQLite-net-pcl in modo semplice ed efficace all'interno di applicazioni sviluppate in C# e .NET MAUI.

5.1 Sprint

Per organizzare lo sviluppo del progetto OraX è stata adottata una metodologia Agile basata sugli Sprint. Uno Sprint rappresenta un periodo di tempo definito e limitato durante il quale il team sviluppa e completa un insieme specifico di funzionalità del progetto. Questa modalità di lavoro consente di suddividere il processo di sviluppo in fasi più piccole e controllabili, facilitando la pianificazione, il monitoraggio dell'avanzamento e la gestione delle eventuali problematiche.

Ogni Sprint prevede:

- la definizione degli obiettivi da raggiungere;
- l'implementazione delle funzionalità previste;
- la verifica dei risultati ottenuti;
- il collegamento tra requisiti funzionali e user stories.

Nel progetto OraX, lo sviluppo è stato suddiviso in tre Sprint principali, ciascuno della durata di tre settimane.

5.1.1 Primo Sprint

Il primo Sprint è dedicato allo sviluppo delle funzionalità fondamentali del sistema, necessarie per consentire all'utente di accedere all'applicazione e personalizzare il proprio ambiente.

5.1.1.1 Requisiti

- REQ-01 – Registrazione al calendario
- REQ-02 – Accesso al proprio profilo
- REQ-03 – Visualizzazione del proprio calendario
- REQ-07 – Personalizzazione del proprio calendario

5.1.1.2 User Stories

- US-01 – Registrazione e accesso dell'utente
- US-04 – Personalizzazione del calendario

5.1.1.3 Obiettivi dello Sprint

Gli obiettivi principali del primo Sprint sono:

- Implementare il sistema di autenticazione;
- Consentire la registrazione degli utenti;
- Permettere l'accesso al calendario personale;
- Sviluppare le funzionalità di personalizzazione grafica dell'applicazione.

5.1.2 Secondo Sprint

Il secondo Sprint si concentra sulla gestione delle attività all'interno del calendario, rappresentando il nucleo operativo dell'applicazione.

5.1.2.1 Requisiti

- REQ-04 – Inserimento di un'attività



- REQ-05 – Informazioni sulle attività
- REQ-06 – Eliminazione e modifica di un'attività
- REQ-11 – Ricerca delle attività

5.1.2.2 User Stories

- US-02 – Creazione e gestione delle attività
- US-03 – Modifica ed eliminazione delle attività
- US-07 – Ricerca delle attività

5.1.2.3 Obiettivi dello Sprint

Gli obiettivi principali del secondo Sprint sono:

- Sviluppare la gestione completa delle attività;
- Permettere la modifica e l'eliminazione degli impegni;
- Implementare il sistema di ricerca;
- Migliorare l'interazione dell'utente con il calendario.

5.1.3 Terzo Sprint

Il terzo Sprint introduce le funzionalità avanzate dell'applicazione, legate a notifiche, statistiche e collaborazione tra utenti.

5.1.3.1 Requisiti

- REQ-08 – Notifiche
- REQ-09 – Statistiche
- REQ-12 – Calendario condiviso
- REQ-13 – Riunioni

5.1.3.2 User Stories

- US-05 – Sistema di notifiche
- US-06 – Visualizzazione statistiche
- US-08 – Calendario condiviso
- US-09 – Richieste di condivisione

5.1.3.3 Obiettivi dello Sprint

Gli obiettivi principali del terzo Sprint sono:

- Implementare il sistema di notifiche automatiche;
- Sviluppare la pagina statistiche;
- Introdurre la condivisione del calendario tra utenti;
- Gestire le richieste di collaborazione e condivisione.

5.2 Design dell'architettura del sistema

L'architettura del sistema **OraX** è basata su un'applicazione sviluppata con .NET MAUI ed è organizzata secondo una suddivisione in tre componenti principali:

- **Interfaccia utente (UI):** comprende le principali pagine dell'applicazione, tra cui login, registrazione, calendario, gestione delle attività, statistiche e profilo utente.
- **Logica applicativa:** gestisce le funzionalità principali del sistema, come l'autenticazione degli utenti, la gestione delle attività e il sistema di notifiche.
- **Database:** responsabile della memorizzazione persistente dei dati, implementato tramite SQLite.



Il flusso dei dati segue un modello lineare: l'utente inserisce o modifica le informazioni attraverso l'interfaccia grafica; tali dati vengono elaborati dalla logica applicativa e successivamente salvati nel database. Quando necessario, i dati vengono recuperati e visualizzati nuovamente all'interno dell'interfaccia.

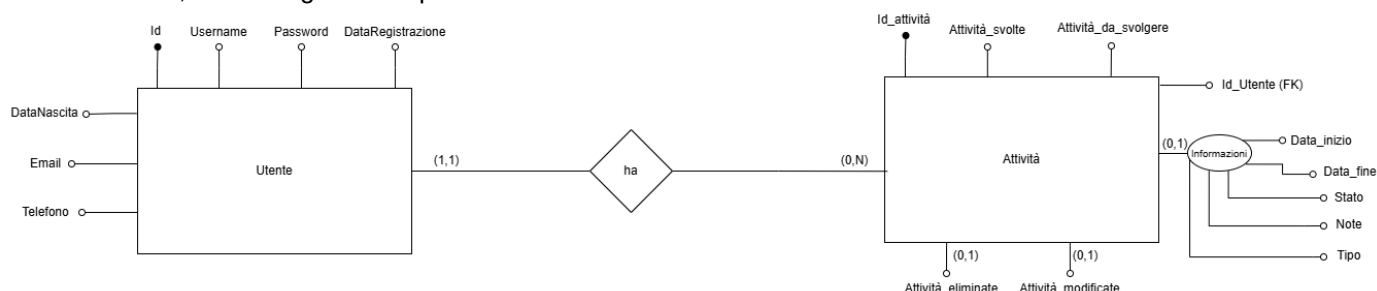


Figura 3 - Diagramma ER

La navigazione dell'applicazione segue il seguente flusso principale:

Login / Registrazione → Home (Calendario) → Dettaglio Attività → Statistiche / Profilo

Questa struttura garantisce una chiara separazione delle responsabilità e facilita sia la manutenzione sia l'estensione futura del sistema.

5.3 Design dei dati e database

Il database è strutturato in modo da gestire le informazioni relative agli utenti e alle attività tramite due tabelle principali:

- **Tabella Utente**

Utilizzata per le funzionalità di login, registrazione e gestione del profilo. Contiene i seguenti campi:

- id (Primary Key)
- username
- password
- data di nascita
- email
- telefono
- data di registrazione
- Tabella Attività

Associata alle funzionalità del calendario e alla gestione delle attività. Contiene i seguenti campi:

- id (Primary Key)
- attività svolte
- attività da svolgere
- attività eliminate
- attività modificate
- stato dell'attività
- note
- tipo di attività
- data di inizio
- data di fine
- id_utente (Foreign Key collegata alla tabella *Utente*)

La relazione tra le due tabelle consente di associare ogni attività a uno specifico utente, garantendo una gestione coerente e organizzata dei dati.

Questa struttura risulta semplice ma efficace, permettendo buone prestazioni e una facile estensione futura del sistema, ad esempio con l'aggiunta di nuove entità o funzionalità.

Figura 4 - Diagramma ER



5.4 Design delle interfacce

La progettazione delle interfacce utente definisce l'aspetto grafico dell'applicazione e le modalità di interazione tra l'utente e il sistema. Le interfacce sono state sviluppate sulla base dei requisiti individuati durante la fase di analisi, sia funzionali che non funzionali, al fine di garantire coerenza con gli obiettivi del progetto.

A supporto della progettazione sono stati realizzati appositi mockup, utilizzati come riferimento durante la fase di sviluppo per visualizzare in modo preliminare la struttura e l'organizzazione delle diverse schermate.

Il design dell'applicazione segue uno stile uniforme e coerente tra le varie pagine, ponendo particolare attenzione a criteri quali leggibilità, semplicità e intuitività. Questo approccio consente di migliorare l'esperienza utente, facilitando la navigazione e rendendo l'utilizzo dell'applicazione immediato ed efficace.

5.4.1 Pagina di login

La **pagina di login** rappresenta il punto di accesso principale dell'applicazione e costituisce la prima interazione tra l'utente e il sistema. L'interfaccia è suddivisa in due sezioni principali.

Nella parte sinistra viene visualizzato un messaggio di benvenuto, con l'obiettivo di accogliere l'utente e invitarlo ad accedere o a registrarsi. Questa sezione ha una funzione principalmente informativa e introduttiva.

Nella parte destra è presente un modulo di autenticazione (form), all'interno del quale l'utente è tenuto a inserire le proprie credenziali, ovvero **username** e **password**, per effettuare l'accesso al sistema.

Nel caso in cui l'utente non disponga ancora di un account, è possibile accedere alla pagina di registrazione selezionando il collegamento "Registrati", che consente la navigazione verso la relativa schermata dedicata alla creazione di un nuovo profilo.

The mockup shows a web browser window titled "Pagina Login". The page is split into two main sections. The left section has a dark purple background and contains the "OraX" logo at the top, followed by the text "Benvenuto/a su OraX" in large white font, and "Accedi o diventa un membro di OraX" in smaller white font below it. The right section has a white background and is titled "Login". It contains two input fields: "Username" and "Password". Below the "Password" field is a purple button labeled "Accedi". At the bottom of the right section, there is a link that says "Non hai un account? Registrati", where "Registrati" is in purple.

Figura 5 - Pagina Login Mockup



5.4.2 Pagina di registrazione

La **pagina di registrazione** consente agli utenti di creare un nuovo account per accedere alle funzionalità dell'applicazione. Essa rappresenta il punto di ingresso per i nuovi utilizzatori del sistema.

L'interfaccia è progettata in modo chiaro e intuitivo e presenta un modulo (form) nel quale l'utente deve inserire i dati necessari alla creazione del proprio profilo. Tra le informazioni richieste rientrano generalmente **nome, cognome, email, numero di telefono, data di nascita, password**.

Al termine della compilazione, l'utente può confermare la registrazione tramite un apposito pulsante, che avvia il processo di validazione e salvataggio dei dati nel database.

La struttura della pagina mantiene coerenza grafica e funzionale con il resto dell'applicazione, garantendo

The screenshot shows a web browser window titled "Pagina Registrazione". The page is split into two main sections. The left section has a dark purple background with the "OraX" logo at the top. Below the logo, it says "Benvenuto/a su OraX" in large white text, and "Registrati per diventare nostro membro" in smaller white text below it. The right section has a white background and is titled "Registrazione". It contains a form with the following fields: "Nome" (text input), "Cognome" (text input), "Email" (text input), "Numero di telefono" (text input), "Data di nascita" (date picker showing "04/17/2026"), and "Password" (text input). At the bottom of the form is a blue button labeled "Registrati".

Figura 6 - Pagina Registrazione

5.4.3 Pagina home

La **pagina Home** rappresenta la sezione centrale dell'applicazione e ospita la funzionalità principale del sistema, ovvero la gestione del calendario e delle attività.

Nella parte superiore sinistra è presente una barra di navigazione che consente all'utente di accedere rapidamente alle diverse sezioni dell'applicazione.

L'interfaccia è suddivisa in due aree principali. Sul lato sinistro è visualizzato il calendario, organizzato per giorni della settimana. Alcuni giorni possono essere contrassegnati da un indicatore (ad esempio un punto), che segnala la presenza di una o più attività associate a quella data.

Sul lato destro è presente l'elenco delle attività, attraverso il quale l'utente può visualizzare le attività pianificate e gestirle. In questa sezione è inoltre disponibile la possibilità di aggiungere nuove attività.

Nella parte superiore destra è collocata una barra di ricerca, che consente di filtrare le attività, migliorando così l'efficienza nella consultazione e nella gestione delle informazioni.

L'organizzazione della pagina è stata progettata per offrire una visione chiara e immediata delle attività, favorendo un'interazione semplice e intuitiva da parte dell'utente.

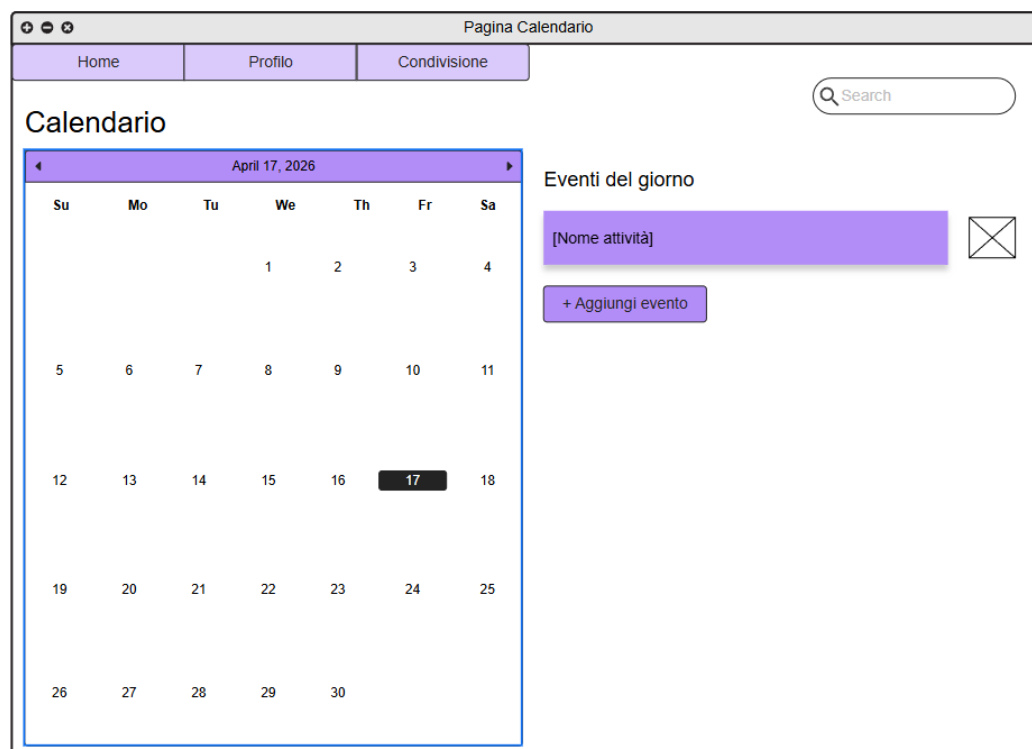


Figura 8 - Pagina Calendario Mockup

5.4.4 Pagina dettagli attività

All'interno della **pagina del calendario**, è prevista un'interazione avanzata con le attività. Se l'utente seleziona un'attività presente nell'elenco, viene visualizzata una finestra modale (pop-up) che consente di consultare e gestire i dettagli associati.

All'interno di questa finestra vengono mostrate tutte le informazioni rilevanti dell'attività, tra cui **priorità, luogo, categoria, data di scadenza e note**.

La finestra modale include inoltre due pulsanti principali che permettono all'utente di eseguire operazioni sull'attività selezionata:

- un pulsante per il **salvataggio delle modifiche**, nel caso in cui vengano apportate variazioni ai dati;
- un pulsante per l'**eliminazione dell'attività**, che consente di rimuoverla dal sistema.

Questa funzionalità permette una gestione completa e immediata delle attività, migliorando l'usabilità dell'applicazione e rendendo più efficiente l'interazione dell'utente con il sistema.

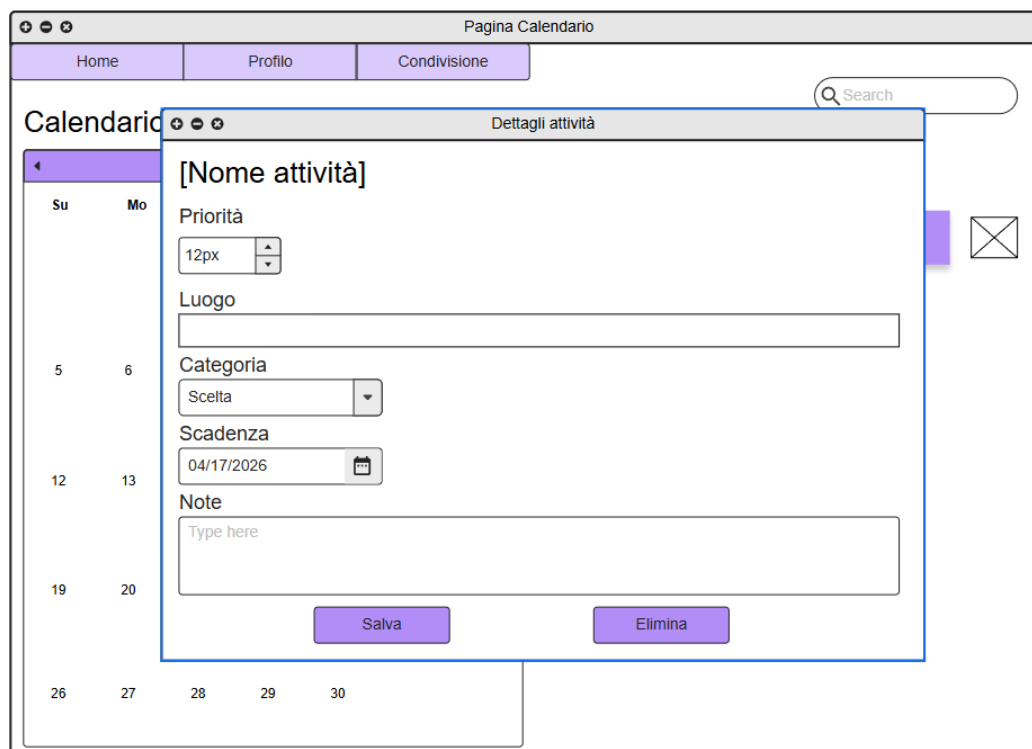


Figura 9 - Pagina Attività Mockup

5.4.5 Notifiche

Una volta effettuato l'accesso all'applicazione, nella pagina del calendario viene visualizzata una finestra modale (pop-up) che richiede all'utente il consenso per l'attivazione delle notifiche.

Questa richiesta ha lo scopo di permettere all'applicazione di inviare promemoria relativi alle attività pianificate, migliorando così l'efficacia nella gestione degli impegni.

L'utente ha la possibilità di accettare o rifiutare l'attivazione delle notifiche tramite appositi comandi presenti nella finestra. La scelta effettuata viene registrata e utilizzata dal sistema per determinare il comportamento futuro dell'applicazione in relazione all'invio delle notifiche.

Questa funzionalità contribuisce a rendere l'applicazione più interattiva e orientata al supporto dell'utente nella gestione delle proprie attività.

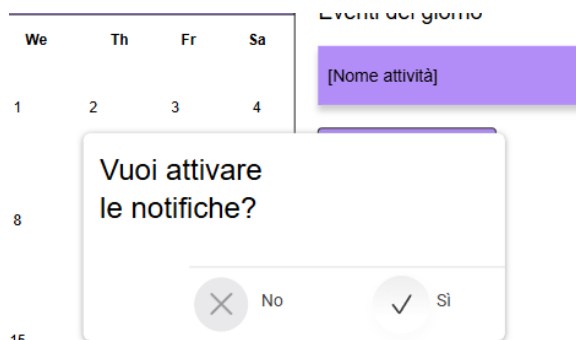


Figura 10 - Notifiche Mockup

5.4.6 Pagina del profilo

La **pagina Profilo** consente all'utente di visualizzare e gestire le proprie informazioni personali all'interno dell'applicazione.

Nella parte superiore è presente una barra di navigazione che permette di spostarsi tra le principali sezioni del sistema, come Home, Profilo e Condivisione.

Subito sotto è disponibile un'area dedicata alla personalizzazione del calendario, tramite un menu a tendina che consente di selezionare un tema o una configurazione predefinita (ad esempio "Default").

La sezione centrale della pagina è dedicata alle informazioni dell'utente. Essa mostra un'area profilo contenente l'immagine utente, il nome e l'indirizzo email, offrendo una visione sintetica ma completa dei dati principali dell'account.

Più in basso è presente una sezione relativa alle attività, suddivisa in due categorie principali: **attività completate** e **attività in sospeso**, permettendo così una rapida consultazione dello stato generale del lavoro dell'utente.

Nella parte inferiore della pagina sono presenti tre pulsanti principali che consentono di eseguire azioni fondamentali:

- **Modifica profilo**, che permette di aggiornare i dati personali dell'utente;
- **Visualizza statistiche**, che consente l'accesso a dati e informazioni relative all'andamento delle attività;
- **Esci dal profilo**, che permette di effettuare il logout dall'applicazione.

Questa struttura è stata progettata per garantire una gestione chiara e intuitiva del profilo utente, integrando sia la consultazione dei dati sia le principali operazioni di gestione dell'account.

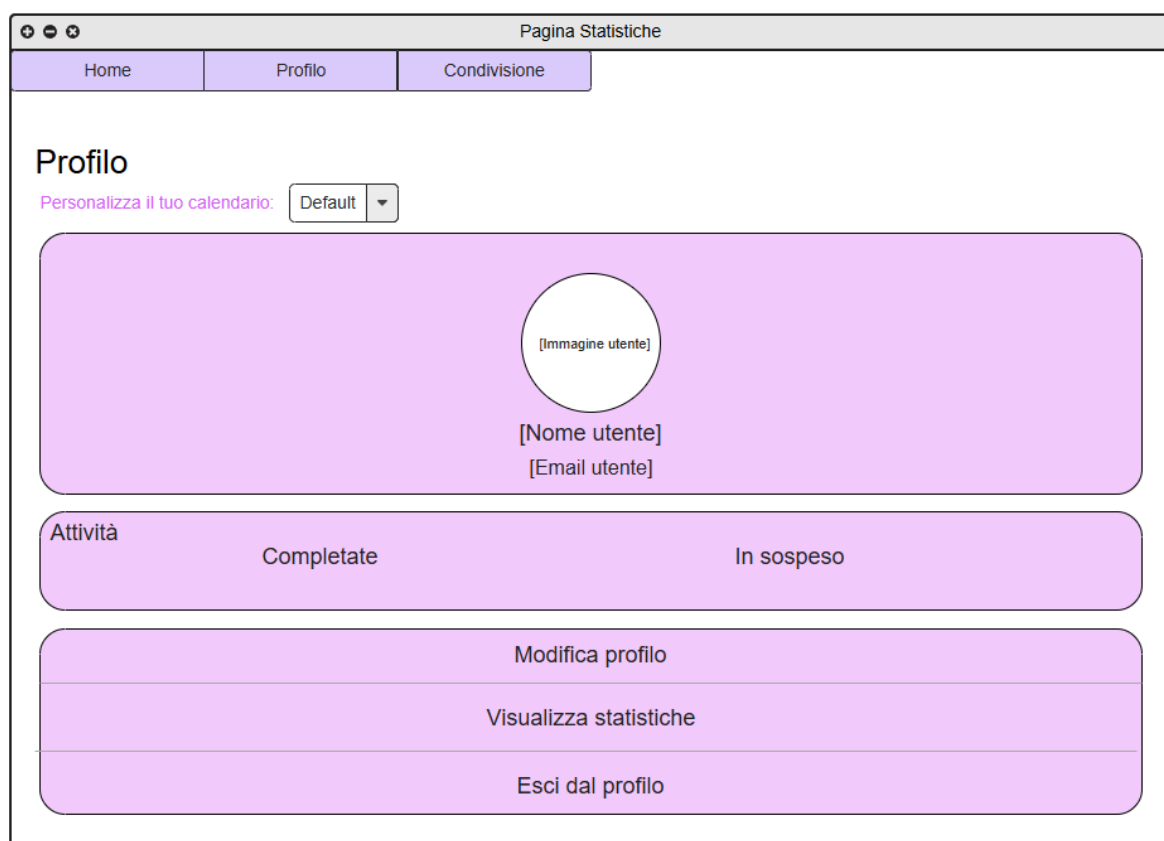


Figura 11 - Pagina Profilo Mockup

5.4.7 Pagina modifiche

La **pagina Modifica Profilo** consente all'utente di aggiornare e personalizzare le proprie informazioni personali all'interno dell'applicazione.

Nella parte superiore della schermata è presente un pulsante di navigazione “Back”, che permette di tornare alla pagina precedente senza salvare eventuali modifiche.

Subito sotto è visualizzata l'immagine del profilo utente, accompagnata da un'opzione che consente di **cambiare la foto profilo**, rendendo possibile la personalizzazione dell'account.

La pagina è suddivisa in diverse sezioni organizzate in modo chiaro e strutturato:

- **Informazioni personali:** contiene i campi modificabili relativi a Nome, Cognome e Username.
- **Contatti:** include i campi per Email e Numero di telefono, utili per la gestione dei dati di contatto dell'utente.
- **Extra:** comprende informazioni aggiuntive come la biografia (Bio) e la data associata al profilo.

Tutti i campi sono editabili, consentendo all'utente di aggiornare i propri dati in qualsiasi momento.

Nella parte inferiore della pagina è presente il pulsante “**Salva modifiche**”, che permette di confermare e memorizzare le modifiche effettuate. Una volta premuto, i dati vengono aggiornati nel sistema e salvati nel database.

Questa schermata è stata progettata per garantire una gestione semplice, ordinata e intuitiva delle informazioni personali, mantenendo coerenza con lo stile grafico dell'intera applicazione.

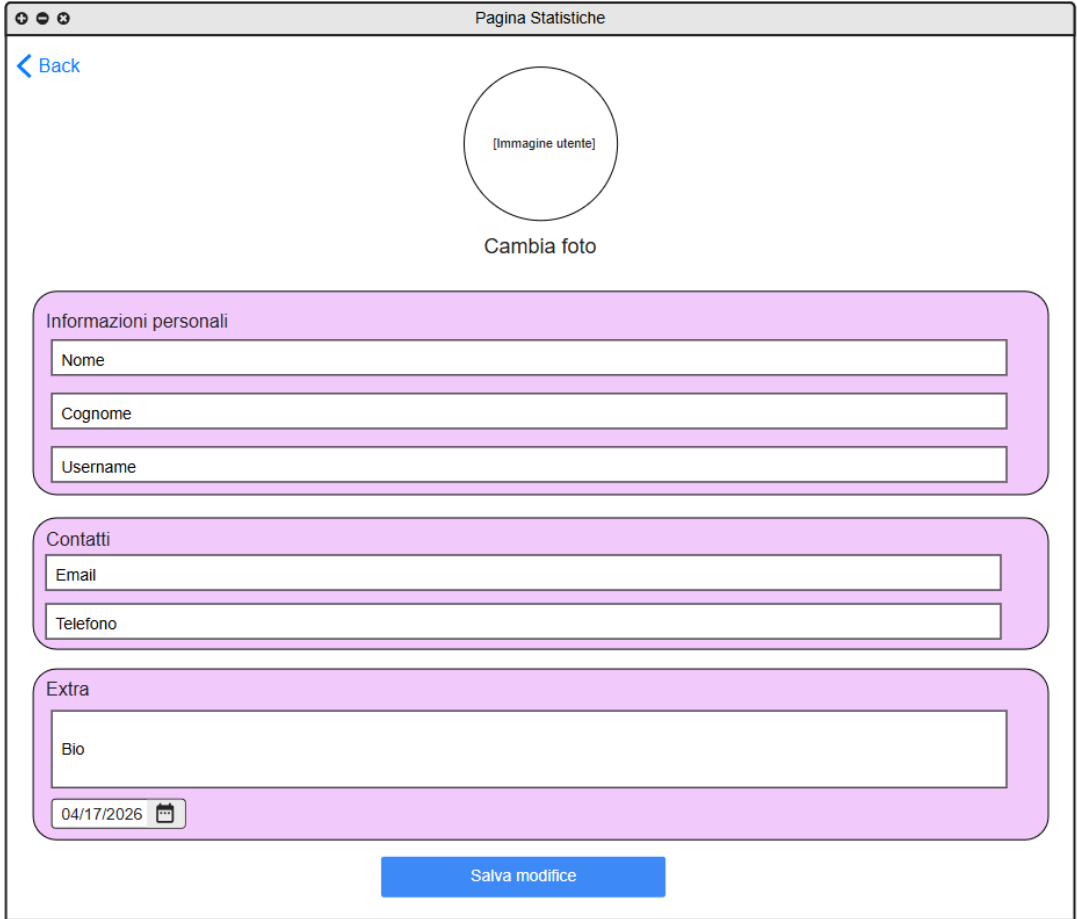


Figura 12 - Pagina Modifiche Mockup



5.4.8 Pagina Statistiche

La pagina **Statistiche** consente all'utente di visualizzare un riepilogo generale delle proprie attività e dei propri progressi all'interno dell'applicazione.

Nella parte superiore della schermata è presente un pulsante di navigazione “Back”, che permette di tornare alla pagina precedente. Accanto ad esso è visualizzato il titolo della pagina, “Pagina Statistiche”, che identifica chiaramente la sezione corrente.

Subito sotto è presente il titolo principale “Le tue statistiche”, che introduce il contenuto della schermata.

La prima sezione è dedicata al riepilogo generale delle attività ed è organizzata in un riquadro suddiviso in tre aree informative:

- **Complete:** indica il numero o la quantità di attività completate.
- **In sospeso:** mostra le attività ancora da completare.
- **Totale:** rappresenta il totale complessivo delle attività.

Questa suddivisione consente all'utente di avere una panoramica immediata del proprio stato di avanzamento.

Segue la sezione **Comportamento**, che include una barra di progresso orizzontale. Tale barra rappresenta visivamente la percentuale di completamento delle attività rispetto al totale. La presenza dell'indicazione percentuale rende la lettura del dato più chiara e immediata, combinando rappresentazione grafica e valore numerico.

Nella parte inferiore della pagina è presente la sezione **Attività recenti**, strutturata in tre riquadri distinti:

- **Oggi:** mostra le attività svolte nella giornata corrente.
- **Ieri:** riporta le attività completate il giorno precedente.
- **2 giorni fa:** visualizza le attività effettuate due giorni prima.

Questa organizzazione temporale consente all'utente di monitorare facilmente la propria continuità e frequenza di utilizzo dell'applicazione.

L'interfaccia è strutturata in modo chiaro e lineare, con sezioni ben separate e contenitori visivamente distinti, così da facilitare la consultazione delle informazioni e garantire un'esperienza utente semplice e intuitiva.

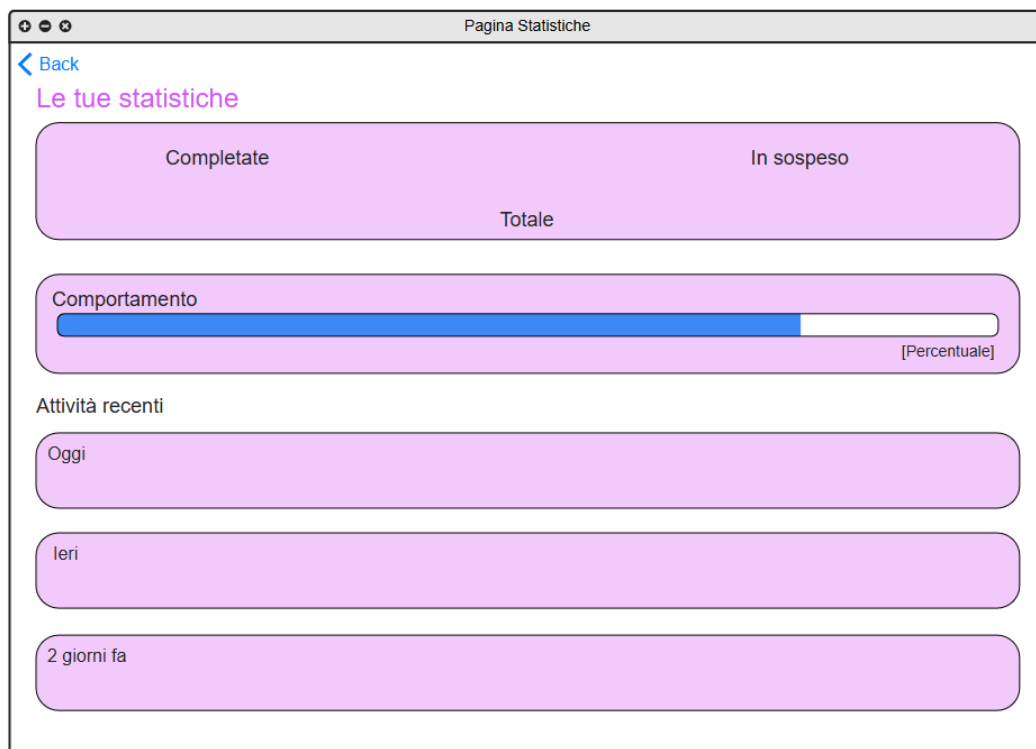


Figura 13 - Pagina Statistiche Mockup

5.4.9 Pagina condivisioni

La pagina **Condivisioni** è accessibile dalla barra di navigazione principale e consente all'utente di creare una condivisione del calendario con un altro utente dell'applicazione.

L'interfaccia è progettata in modo semplice ed essenziale, così da rendere immediata l'operazione di condivisione. La schermata prevede l'inserimento di due informazioni principali:

- **Username:** campo in cui l'utente inserisce il nome utente della persona con cui desidera condividere il calendario.
- **Nome della categoria:** campo in cui viene specificata la categoria di attività che si intende condividere.

Una volta completati i campi e confermata l'operazione, il sistema registra la condivisione tra i due utenti.

Il meccanismo di funzionamento prevede che, al momento della creazione di una nuova attività, selezionando la categoria precedentemente impostata per la condivisione, l'attività venga automaticamente resa visibile anche all'altro utente coinvolto.

In questo modo, la pagina permette di gestire in maniera mirata la condivisione delle attività, limitandola a specifiche categorie e garantendo un maggiore controllo sulle informazioni condivise.

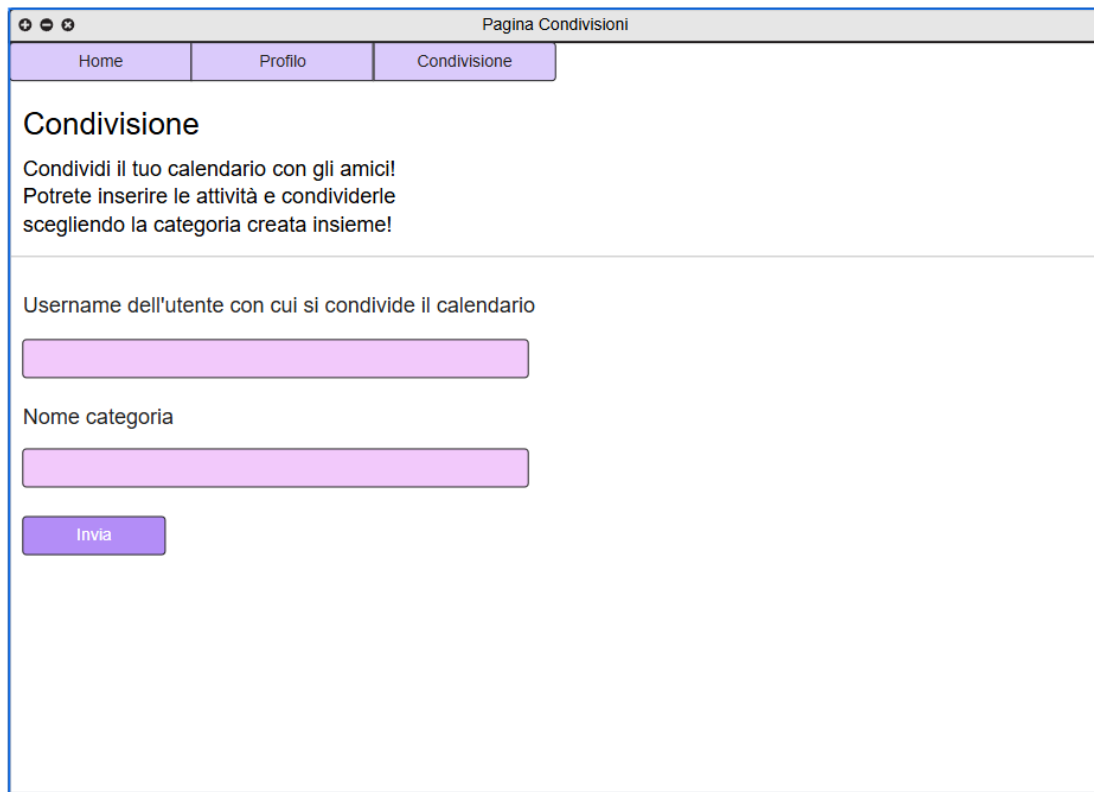


Figura 14 - Pagina Condivisioni Mockup

5.4.10 Notifica condivisione

Quando un utente invia una richiesta di condivisione a un altro utente, il sistema genera automaticamente una notifica destinata al destinatario della richiesta.

A questo punto, ha la possibilità di scegliere tra due opzioni:

- Accettare la richiesta, confermando la condivisione della categoria indicata. In questo caso, le attività create all'interno di quella categoria saranno visibili a entrambi gli utenti.
- Rifiutare la richiesta, annullando il processo di condivisione e impedendo qualsiasi accesso alle proprie attività.

Questo meccanismo garantisce un maggiore controllo e tutela della privacy, poiché la condivisione diventa effettiva solo dopo l'approvazione esplicita del destinatario.

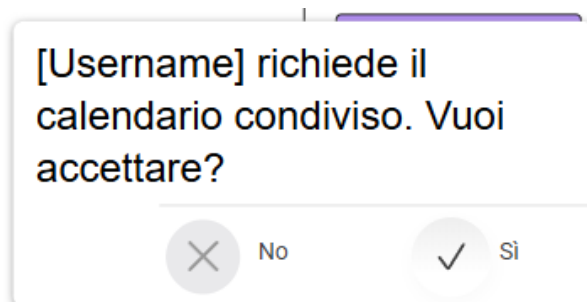


Figura 15 - Notifica Condivisione Mockup



5.5 Design procedurale

Il design procedurale descrive in modo dettagliato il comportamento interno del sistema OraX, definendo i flussi operativi principali, le modalità di elaborazione dei dati e l'interazione tra le diverse componenti software.

Questa sezione ha lo scopo di tradurre i requisiti funzionali in processi concreti, evidenziando come il sistema realizza le funzionalità previste, mantenendo coerenza con l'architettura definita nei capitoli precedenti.

5.5.1 Flusso generale dell'applicazione

Il diagramma di flusso rappresenta il funzionamento generale dell'applicazione OraX, illustrando le principali operazioni che l'utente può svolgere e le decisioni che il sistema deve gestire.

Il processo ha inizio con la verifica della presenza di un account. Se l'utente possiede già un account, può procedere con l'accesso inserendo le proprie credenziali. In caso contrario, viene indirizzato alla fase di registrazione. Il sistema verifica la validità dei dati inseriti e, in caso di errore, restituisce un messaggio appropriato.

Una volta effettuato l'accesso con successo, l'utente accede alla pagina principale del calendario. A questo punto, il sistema può richiedere l'autorizzazione per l'invio delle notifiche.

Dal calendario, l'utente può eseguire diverse operazioni principali:

- **Creazione di una nuova attività**, inserendo le relative informazioni
- **Modifica di un'attività esistente**, con possibilità di salvare o annullare le modifiche
- **Eliminazione di un'attività**
- **Ricerca di attività specifiche**
- **Visualizzazione del profilo utente**
- **Accesso alle statistiche**

Nel caso della modifica del profilo, l'utente può aggiornare i propri dati e decidere se salvare o meno le modifiche.

Il sistema prevede inoltre una funzionalità di **condivisione**, che consente all'utente di inviare una richiesta ad un altro utente. Il destinatario può accettare o rifiutare la richiesta, determinando così l'esito della condivisione.

Il processo termina quando l'utente conclude le operazioni, portando allo stato finale del sistema.

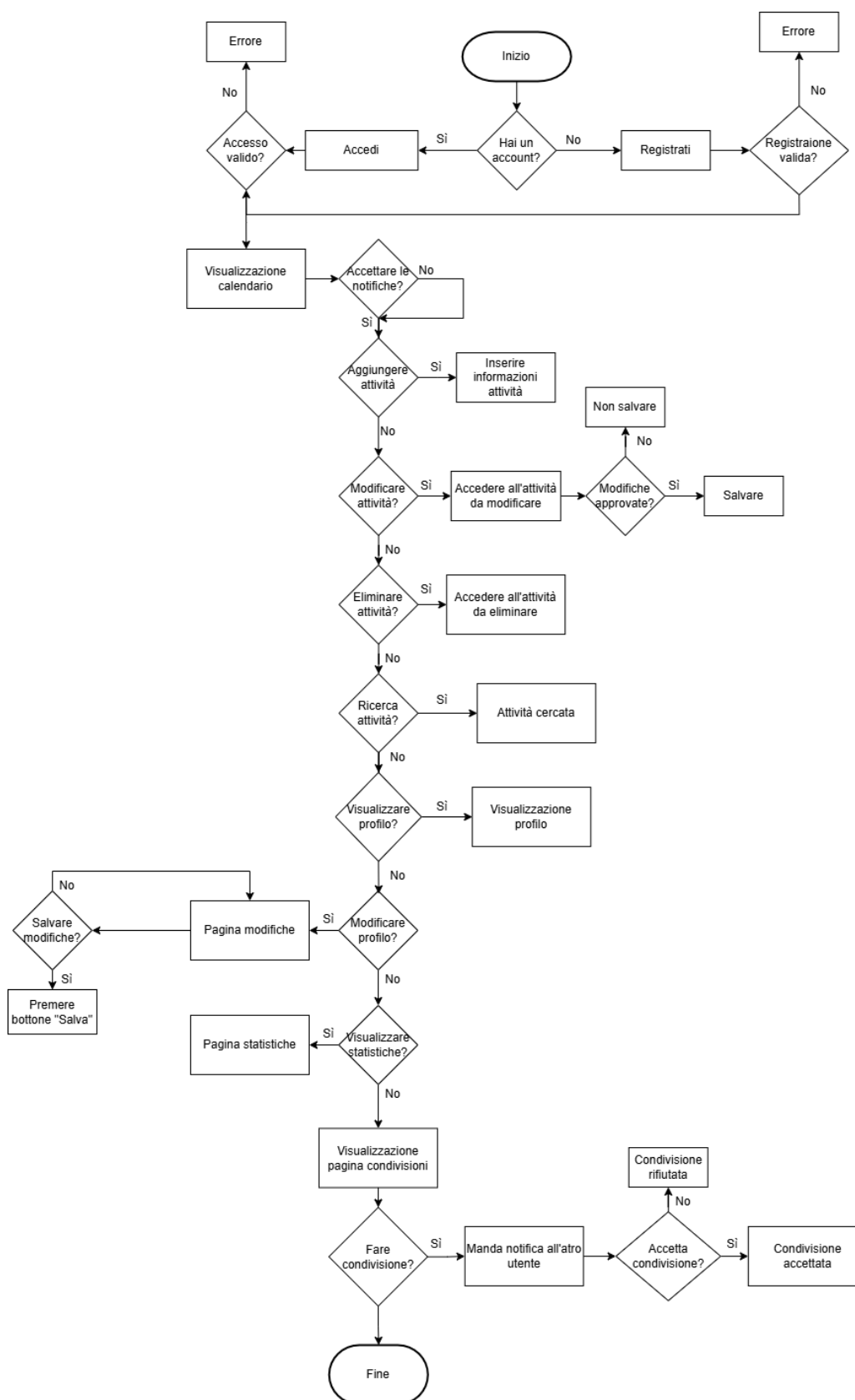


Figura 16 - Diagramma di flusso



5.5.2 Gestione dell'autenticazione

5.5.2.1 Login

Il processo di autenticazione avviene secondo i seguenti passaggi:

1. L'utente inserisce username e password
2. Il sistema verifica le credenziali nel database
3. Se i dati sono validi:
 - viene avviata la sessione utente
 - viene caricata la pagina principale (Home)
4. In caso contrario:
 - viene restituito un messaggio di errore

5.5.2.2 Registrazione

- 1) L'utente inserisce i dati richiesti nel form
- 2) Il sistema valida i dati (formato email, password, campi obbligatori)
- 3) I dati vengono salvati nel database
- 4) L'utente può accedere al sistema tramite login

5.5.3 Gestione delle attività

La gestione delle attività rappresenta la funzionalità principale del sistema.

5.5.3.1 Creazione attività

1. L'utente seleziona un giorno nel calendario
2. Inserisce i dettagli (titolo, data, categoria, note...)
3. Il sistema valida i dati
4. L'attività viene salvata nel database
5. Il calendario viene aggiornato

5.5.3.2 Modifica attività

1. L'utente seleziona un'attività
2. Modifica i dati nel popup
3. Il sistema aggiorna il database
4. L'interfaccia si aggiorna automaticamente

5.5.3.3 Eliminazione attività

1. L'utente preme "Elimina"
2. Il sistema rimuove l'attività dal database
3. L'attività scompare dalla UI

5.5.4 Sistema di notifiche

Il sistema di notifiche consente di avvisare l'utente prima della scadenza delle attività.

Flusso operativo:

1. L'utente attiva/disattiva le notifiche
2. Per ogni attività viene salvato un tempo di preavviso
3. Il sistema monitora le scadenze
4. Al momento opportuno:



- viene mostrata una notifica popup

5.5.5 Sistema di ricerca

Il sistema di ricerca consente di individuare rapidamente le attività.

1. L'utente inserisce una parola chiave
2. Il sistema esegue una query nel database
3. I risultati vengono filtrati
4. Le attività trovate vengono visualizzate
5. L'utente può selezionare un risultato per visualizzarne i dettagli

5.5.6 Gestione delle statistiche

Le statistiche permettono all'utente di analizzare la propria produttività.

1. Il sistema aggiorna i dati ad ogni modifica delle attività
2. Vengono calcolati:
 - numero di attività completate
 - attività in sospeso
 - totale attività
3. Viene calcolata la percentuale di completamento
4. I dati vengono visualizzati nella pagina statistiche

5.5.7 Gestione della condivisione

Il sistema permette la condivisione delle attività tra utenti.

1. L'utente invia una richiesta di condivisione
2. Il destinatario riceve una notifica
3. Il destinatario può:
 - accettare → attiva la condivisione
 - rifiutare → annulla il processo
4. Le attività condivise diventano visibili a entrambi

5.5.8 Struttura logica delle componenti

La logica applicativa è organizzata in moduli principali:

- **Gestione utenti** → autenticazione e profili
- **Gestione attività** → CRUD delle attività
- **Gestione notifiche** → promemoria automatici
- **Gestione statistiche** → analisi dati
- **Gestione condivisione** → collaborazione tra utenti
- **Database service** → accesso e gestione SQLite

Questa suddivisione garantisce modularità e facilita eventuali estensioni future.

5.5.9 Gestione degli errori

Il sistema implementa controlli per garantire affidabilità:

- validazione input utente
- gestione errori database
- controllo operazioni non valide

In caso di errore:



- viene mostrato un messaggio chiaro all'utente
- il sistema evita interruzioni improvvise

5.5.10 Considerazioni finali

Il design procedurale adottato consente di:

- tradurre i requisiti in processi concreti
- mantenere una struttura chiara e modulare
- garantire facilità di manutenzione
- supportare l'evoluzione futura del sistema

L'organizzazione dei flussi e delle componenti rende il sistema efficiente, scalabile e coerente con gli obiettivi del progetto.



6 Implementazione

6.1 Creazione dell'applicativo

La fase di implementazione rappresenta il momento in cui tutto ciò che era stato progettato nella fase di analisi e progettazione viene trasformato in un'applicazione concreta e funzionante. Durante questa fase sono state sviluppate tutte le componenti software dell'applicazione OraX utilizzando il framework .NET MAUI, il linguaggio C# e interfacce grafiche realizzate in XAML.

La scelta di utilizzare .NET MAUI è stata fatta perché permette di creare applicazioni moderne multiplatforma utilizzando un'unica codebase. Questo ha reso possibile sviluppare l'applicazione mantenendo una struttura ordinata e facilmente espandibile.

L'obiettivo principale dell'implementazione era quello di realizzare un'applicazione moderna per la gestione di attività e calendari condivisi, con un'interfaccia semplice da utilizzare ma allo stesso tempo completa di funzionalità avanzate come:

- gestione utenti;
- login;
- calendari condivisi;
- notifiche;
- statistiche;
- personalizzazione grafica.

Durante lo sviluppo il progetto è stato migliorato più volte rispetto alla progettazione iniziale. Alcune funzionalità sono state completamente riscritte per migliorarne la stabilità e l'esperienza utente.

L'applicazione finale è stata organizzata utilizzando una struttura modulare. Ogni parte del progetto ha una responsabilità specifica:

- le pagine XAML si occupano dell'interfaccia grafica;
- i file .xaml.cs gestiscono la logica delle pagine;
- i servizi (Services) si occupano della gestione database e notifiche;
- i modelli (Models) rappresentano i dati dell'applicazione.

Questa organizzazione ha permesso di mantenere il codice più leggibile e facilmente modificabile.

6.2 Struttura generale

Il progetto è stato organizzato seguendo una struttura ordinata, in modo da separare correttamente la logica applicativa dall'interfaccia grafica.

Le principali cartelle utilizzate sono:

- Models, contenente le classi utilizzate per rappresentare i dati;
- Services, utilizzata per la gestione del database e delle notifiche;
- Pages, contenente le pagine XAML dell'applicazione;
- Resources, utilizzata per immagini, temi e icone.

Questa organizzazione ha reso il progetto più leggibile e facilmente modificabile durante tutto lo sviluppo.

La configurazione principale dell'applicazione viene eseguita nel file App.xaml.cs, nel quale vengono inizializzati i servizi necessari al corretto funzionamento dell'app.

```
public partial class App :  
    Application  
{  
    public App()  
    {  
        InitializeComponent();  
  
        MainPage = new AppShell();  
    }  
}
```



6.3 Database SQLite

Per la gestione dei dati è stato utilizzato SQLite. La scelta di questo database è stata fatta perché SQLite risulta molto leggero, veloce e facilmente integrabile all'interno di un progetto .NET MAUI senza richiedere server esterni.

Il database viene inizializzato automaticamente all'avvio dell'applicazione tramite il servizio DatabaseService. Questo servizio si occupa della creazione delle tabelle, dell'inserimento dei dati, dell'aggiornamento delle informazioni e della lettura delle attività e dei calendari salvati dagli utenti.

Durante l'inizializzazione vengono create automaticamente tutte le tabelle necessarie al corretto funzionamento dell'applicazione.

```
await database.CreateTableAsync<UserDb>();  
await database.CreateTableAsync<CalendarioDb>();  
await database.CreateTableAsync<AttivitaDb>();
```

Questa soluzione permette all'applicazione di creare automaticamente il database locale senza richiedere configurazioni manuali all'utente.

Durante lo sviluppo del progetto sono state aggiunte più volte nuove colonne alle tabelle esistenti. Per evitare di eliminare il database e perdere i dati già salvati, è stato implementato un sistema che controlla automaticamente se una colonna esiste già.

```
await AggiungiColonnaSeManca(  
    "AttivitaDb",  
    "NotificaInviata",  
    "INTEGER DEFAULT 0"  
);
```

Questo approccio ha permesso di aggiornare il database durante lo sviluppo mantenendo la compatibilità con le versioni precedenti dell'applicazione.

6.4 Pagina di login

Le pagine di login e registrazione della nostra applicazione OraX permettono agli utenti del calendario di accedere con un account personale.

I dati vengono salvati tramite un database SQLite. Questa parte del progetto è composto da tre aree principali: i modelli (la struttura dei dati), i servizi (la logica e accesso al database) e le pagine (l'interfaccia utente).

6.4.1 User.cs

Questa classe rappresenta la "forma" di un utente nel sistema. Ogni volta che si vuole salvare o leggere un utente dal database, si usa questo oggetto. Sono gli attributi che ogni utente ha al momento della creazione del suo account, che avrà poi la possibilità di modificare.

```
[PrimaryKey, Unique]  
public string Username { get; set; }  
public string PasswordHash { get; set; }
```

L'attributo **[PrimaryKey, Unique]** dice a SQLite che l' Username è l'identificatore univoco di ogni utente: non possono esistere due righe nel database con lo stesso valore. La password non viene mai salvata in chiaro — il campo si chiama PasswordHash proprio perché conterrà il risultato di una funzione crittografica. Abbiamo poi gli attributi Nome, Cognome, DataNascita, Email, Telefono e DataRegistrazione.



6.4.2 Sicurezza – PasswordHelper.cs

Questo servizio ha un unico compito: trasformare una password in chiaro in un hash crittografico. Viene usato sia in fase di registrazione (per salvare l'hash) sia in fase di login (per confrontare l'hash della password digitata con quello salvato).

```
public static string HashPassword(string password)
{
    using var sha = SHA256.Create();
    byte[] bytes =
        Encoding.UTF8.GetBytes(password);
    byte[] hash = sha.ComputeHash(bytes);
    return Convert.ToBase64String(hash);
}
```

Il processo si attua in tre passi: la stringa viene convertita in byte, poi questi byte vengono trasformati dall'algoritmo **SHA-256** in un hash di 256 bit, e infine l'hash viene codificato in **Base64** per poterlo salvare come stringa di testo.

SHA-256 è una funzione a senso unico: non sarà quindi mai possibile risalire alla password originale partendo dall'hash, il che rende i dati al sicuro anche se il database venisse letto.

6.4.3 Database – DatabaseService.cs

Questo servizio gestisce tutta la comunicazione con il database SQLite locale. Ci sono tre operazioni: inizializzare il database, registrare un nuovo utente e cercare un utente (per il login o per controllare se uno username esiste già). Tutte le operazioni sono **asincrone** per non bloccare l'interfaccia mentre si accede.

```
public async Task Init()
{
    if (database != null) return;

    string dbPath =
        Path.Combine(FileSystem.AppDataDirectory,
            "orax.db");
    database = new
        SQLiteAsyncConnection(dbPath);
    await database.CreateTableAsync<User>();
}
```

Il controllo iniziale `if (database != null) return` implementa il pattern **lazy initialization**: la connessione viene creata solo la prima volta che serve, e le volte successive viene riutilizzata quella già esistente. Il file `orax.db` viene salvato nella cartella privata dell'app sul dispositivo, non accessibile da altre applicazioni (`C:\Users\gwj832\AppData\Local\User Name\com.companyname.orax\Data`).



6.4.3.1 Codice – Login

```
public async Task<User> GetUser(string
username, string password)
{
    await Init();
    return await database.Table<User>()
        .Where(u => u.Username == username &&
u.PasswordHash == password)
        .FirstOrDefaultAsync();
}
```

La query cerca un utente che corrisponda sia all'username che all'hash della password. Se non esiste, `FirstOrDefaultAsync()` restituisce null, che viene poi interpretato dalla pagina di login come credenziali errate, quindi da l'errore e lo notifica all'utente.

6.4.4 LoginPage.xaml

È la prima schermata che vede l'utente. È composta da due colonne: a sinistra un pannello grafico con il logo e il nome dell'app, a destra il form con i campi username e password e il pulsante di accesso. Da qui si

```
<Grid ColumnDefinitions="*,*">
    <Grid Grid.Column="0"
        BackgroundColor="#0a0a0f">
        <!-- Pannello sinistro con logo e
        testo -->
    </Grid>
    <ScrollView Grid.Column="1"
        BackgroundColor="#F9F9FB">
        <!-- Form di login -->
    </ScrollView>
</Grid>
```

può anche navigare alla pagina di registrazione o a quella principale del calendario. Il layout usa un Grid a due colonne di uguale larghezza. Il pannello sinistro è puramente decorativo e informativo, quello destro è funzionale e scrollabile per adattarsi a schermi di altezze diverse.



6.4.4.1 Logica

```
private async void OnLoginClicked(object sender, EventArgs e)
{
    if
    (string.IsNullOrWhiteSpace(username) || string.IsNullOrWhiteSpace(
password))
    {
        await DisplayAlert("Errore", "Inserisci username e
password", "OK");
        return;
    }

    string hashPassword = PasswordHelper.HashPassword(password);
    User user = await database.GetUser(username, hashPassword);

    if (user == null)
    {
        await DisplayAlert("Errore", "Username o password
sbagliati", "OK");
        return;
    }

    await DisplayAlert("Login riuscito", "Benvenuto/a " +
user.Nome + user.Cognome, "OK");
}
```

Prima si controlla che i campi non siano vuoti, poi la password viene hashata e confrontata con quella nel database. In caso di credenziali errate il messaggio è generico ("Username o password sbagliati"), non viene specificato quale dei due sia sbagliato, per non dare informazioni utili a chi tenta un accesso non autorizzato.

6.5 Pagina di registrazione

Permette a un nuovo utente di creare un account compilando nome, cognome, email, telefono, data di nascita, username e password (con conferma). Include anche un pulsante per mostrare e nascondere la password. Al termine della registrazione, l'utente viene riportato alla pagina di login.



6.5.1 Toggle – visibilità password

Un booleano tiene traccia dello stato corrente della password. Ad ogni click viene invertito, e di conseguenza si aggiorna sia la proprietà `IsPassword` del campo (che nasconde o fa vedere il testo) sia l'icona del pulsante, dando un anche un feedback visibile all'utente.

```
private void OnTogglePasswordClicked(object sender, EventArgs e)
{
    passwordVisibile = !passwordVisibile;
    PasswordEntry.IsPassword =
    !passwordVisibile;
    OcchioPasswordButton.Source =
    passwordVisibile
        ? "occhio_aperto.png"
        : "occhio_chiuso.png";
}
```



6.5.2 Logica

```
private async void OnRegisterClicked(object sender, EventArgs e)
{
    // tutti i campi compilati?
    if (string.IsNullOrWhiteSpace(NomeEntry.Text) || ...)
    {
        await DisplayAlert("Errore", "Compila tutti i campi", "OK");
        return;
    }

    // le due password coincidono?
    if (PasswordEntry.Text != PasswordEntry2.Text)
    {
        await DisplayAlert("Errore", "Le password non coincidono",
"OK");
        return;
    }

    // lo username è già in uso?

    var userEsistente = await d
        atabase.GetUserByUsername(UsernameEntry.Text);
    if (userEsistente != null)
    {
        await DisplayAlert("Errore", "Username già esistente",
"OK");
        return;
    }

    // creazione e salvataggio
    User nuovoUser = new User
    {
        Nome = NomeEntry.Text,
        ...
        PasswordHash =
PasswordHelper.HashPassword(PasswordEntry.Text),
        DataRegistrazione = DateTime.Now
    };

    await database.RegistraUser(nuovoUser);
    await Navigation.PopAsync();
}
```

La logica va a **step sequenziali con early return**: ogni controllo, se fallisce, mostra subito un errore e interrompe l'esecuzione. In questo modo il codice resta leggibile. I controlli sono ordinati per costo: prima quelli lato UI (veloci, senza nessun accesso al database), poi quelli che richiedono una query (verificare se lo username esiste già), e solo alla fine si crea e salva l'utente.



6.6 Calendario

La sezione calendario rappresenta una delle funzionalità principali dell'applicazione OraX. Questa parte permette all'utente di visualizzare le proprie attività, aggiungere nuovi eventi, cercare attività già salvate, completarle, eliminarle e gestire più calendari. Inoltre, è stata implementata anche la condivisione dei calendari tra utenti tramite un sistema di inviti.

Dal punto di vista tecnico, il calendario è stato realizzato utilizzando .NET MAUI, separando la parte grafica dalla parte logica. La parte grafica si trova nel file:

- CalendarPageV2.xaml;

mentre la parte logica si trova nel file:

- CalendarPageV2.xaml.cs;

Questa separazione permette di mantenere il codice più ordinato: nel file XAML viene definita l'interfaccia visibile all'utente, mentre nel file C# vengono gestiti i dati, gli eventi, il database e le azioni eseguite dall'utente.

6.6.1 Struttura

La pagina del calendario è stata costruita utilizzando una Grid principale. Questa griglia permette di dividere la schermata in più sezioni ordinate. Nella parte superiore si trova l'intestazione con il titolo della pagina e il nome dell'utente collegato. Subito sotto sono presenti la barra di ricerca, il selettore dei calendari, il pulsante per creare un nuovo calendario e il pulsante per invitare altri utenti.

La parte centrale della pagina è divisa in due colonne: a sinistra si trova il calendario vero e proprio, mentre a destra viene mostrata la lista degli eventi del giorno selezionato.

Un esempio della struttura principale è il seguente:



```
<Grid RowDefinitions="Auto,Auto,*">

    <!-- HEADER -->
    <Grid Grid.Row="0">
        <Label Text="Calendario"/>
        <Label x:Name="LabelNomeUtente"
Text="Nome Utente"/>
    </Grid>

    <!-- SEARCH + CALENDARI -->
    <Border Grid.Row="1">
        <VerticalStackLayout>
            <SearchBar
x:Name="SearchBarAttivita"/>
            <Picker x:Name="CalendarioPicker"/>
        </VerticalStackLayout>
    </Border>

    <!-- CONTENUTO -->
    <Grid Grid.Row="2"
ColumnDefinitions="1.15*,0.85*">
        <!-- Calendario -->
        <!-- Lista eventi -->
    </Grid>

</Grid>
```

Questa organizzazione rende la pagina più chiara e permette all'utente di usare tutte le funzioni principali senza cambiare schermata.

6.6.2 Grafica

Per visualizzare il calendario è stato utilizzato il componente `Calendar` del plugin `Plugin.Maui.Calendar`. Questo componente permette di mostrare un calendario mensile e di collegare a ogni giorno una lista di eventi.

Nel file XAML il calendario è stato inserito in questo modo:

```
<controls:Calendar
    Events="{Binding EventiCalendario}"
    EventIndicatorColor="#7C3AED"
    EventsScrollViewVisible="False"
    TodayFillColor="#C4B5FD"
    SelectedDayBackgroundColor="#7C3AED"
    SelectedDate="{Binding DataSelezionata}"
    FirstDayOfWeek="Monday"/>
```

La proprietà `Events` è collegata tramite binding a `EventiCalendario`, cioè la raccolta che contiene tutti gli eventi da mostrare nel calendario. La proprietà `SelectedDate` è collegata invece a `DataSelezionata`, che rappresenta il giorno scelto dall'utente.



Il colore principale scelto per il calendario è il viola #7C3AED, utilizzato sia per il giorno selezionato sia per gli elementi principali dell'interfaccia. Questo permette di mantenere uno stile uniforme con il resto dell'applicazione.

6.6.3 Collegamento tra interfaccia e codice C#

Nel file `CalendarPageV2.xaml.cs` è stata creata la classe `CalendarPageV2`, che eredita da `ContentPage` e implementa `INotifyPropertyChanged`.

```
public partial class CalendarPageV2 :
    ContentPage, INotifyPropertyChanged
{
    readonly DatabaseService database;
    List<Calendario> calendari = new();
    Calendario? calendarioCorrente;
    DateTime dataSelezionata = DateTime.Today;
}
```

La variabile `database` serve per comunicare con il database SQLite tramite il servizio `DatabaseService`. La lista `calendari` contiene tutti i calendari disponibili per l'utente. La variabile `calendarioCorrente` indica il calendario attualmente selezionato. Infine, `dataSelezionata` contiene la data scelta nel calendario. Nel costruttore della pagina viene impostato il `BindingContext`:

```
public
CalendarPageV2(DatabaseService db)
{
    database = db;
    InitializeComponent();
    BindingContext = this;
}
```

Questa istruzione è fondamentale perché permette alla pagina XAML di collegarsi alle proprietà presenti nel file C#. Senza `BindingContext = this`, elementi come `EventiCalendario`, `AttivitaDelGiorno` e `DataSelezionata` non verrebbero aggiornati correttamente nella grafica.

6.6.4 Data selezionata

La data selezionata viene gestita tramite la proprietà `DataSelezionata`.

```
public DateTime DataSelezionata
{
    get => dataSelezionata;
    set
    {
        if (dataSelezionata != value)
        {
            dataSelezionata = value;
            OnPropertyChanged();
            AggiornaAttivita(value);
        }
    }
}
```



Quando l'utente clicca su un giorno diverso del calendario, il valore di `DataSelezionata` cambia. A quel punto viene richiamato il metodo `AggiornaAttivita(value)`, che aggiorna la lista degli eventi mostrati nella parte destra della pagina.

In questo modo il calendario e la lista degli eventi sono collegati: scegliendo un giorno, vengono visualizzate solo le attività appartenenti a quel giorno.

6.6.5 Collezioni usate per mostrare i dati

Per mostrare i dati nella pagina sono state utilizzate diverse collezioni.

```
public ObservableCollection<Attivita> AttivitaDelGiorno { get; set; } = new();
public ObservableCollection<Attivita> RisultatiRicerca { get; set; } = new();
public EventCollection EventiCalendario { get; set; } = new();
public List<Tipo> tuttiTipi { get; set; } = new();
```

- `AttivitaDelGiorno` contiene solo le attività del giorno selezionato. Questa collezione è collegata alla `CollectionView` presente nella parte destra della pagina.
- `RisultatiRicerca` contiene invece le attività trovate quando l'utente usa la barra di ricerca.
- `EventiCalendario` è la collezione usata dal componente calendario per sapere in quali giorni sono presenti eventi.
- `tuttiTipi` contiene i tipi di attività disponibili, ad esempio scuola, casa o viaggi.

6.6.6 Caricamento pagina iniziale

Quando la pagina viene aperta, viene eseguito il metodo `OnAppearing`.

```
protected override async void OnAppearing()
{
    base.OnAppearing();
    LoadTheme();
    await CaricaTipi();
    await CaricaCalendari();

    var user = UserSession.CurrentUser;
    if (user != null)
        LabelNomeUtente.Text = $"{user.Nome} {user.Cognome}";
}
```

Questo metodo svolge diverse operazioni importanti. Prima carica il tema grafico dell'applicazione, poi carica i tipi di attività e infine carica i calendari collegati all'utente. Dopo questo, viene mostrato il nome dell'utente nella parte superiore della pagina.

Questa fase è importante perché prepara tutti i dati necessari prima che l'utente inizi a usare il calendario.

6.6.7 Gestione tipi di attività

I tipi di attività servono per classificare gli eventi. Per esempio, un'attività può essere di tipo "Scuola", "Casa" oppure "Viaggi". Ogni tipo possiede anche un colore.

Il metodo che carica i tipi è il seguente:



```
async Task CaricaTipi()
{
    await database.InitTipiDefault();
    var tipiDb = await database.GetTipi();

    tuttiTipi = tipiDb.Select(t => new Tipo
    {
        Id = t.Id,
        Nome = t.Nome,
        Colore = Color.FromArgb(t.ColoreHex)
    }).ToList();

    OnPropertyChanged(nameof(tuttiTipi));
}
```

Prima viene chiamato `InitTipiDefault()`, che crea i tipi base se non esistono ancora nel database. Successivamente i tipi vengono letti dal database e trasformati in oggetti `Tipo`, utilizzabili dall'interfaccia grafica.

Nel database i colori sono salvati come stringhe esadecimali, ad esempio `#7FFFD4`, perché SQLite non può salvare direttamente un oggetto `Color`. Per questo motivo, quando vengono caricati, vengono convertiti con:

- `Color.FromArgb(t.ColoreHex)`

6.6.8 Creazione e caricamento dei calendari

Il metodo `CaricaCalendari` recupera tutti i calendari dell'utente collegato.

```
async Task CaricaCalendari()
{
    var user = UserSession.CurrentUser;
    if (user == null) return;

    calendari = await database.GetCalendariUtente(user.Username);

    if (!calendari.Any())
    {
        var calendario = new Calendario
        {
            Nome = "Il mio calendario",
            CreatoreUsername = user.Username,
            Condiviso = false
        };

        int calendarioId = await database.SalvaCalendario(calendario);

        await database.AggiungiUtenteCalendario(new CalendarioUtente
        {
            CalendarioId = calendarioId,
            Username = user.Username
        });

        calendari = await database.GetCalendariUtente(user.Username);
    }
}
```



Se l'utente non possiede ancora nessun calendario, il programma ne crea automaticamente uno chiamato "Il mio calendario". Questa scelta migliora l'esperienza utente, perché permette di iniziare subito a usare l'app senza dover creare manualmente il primo calendario.

Dopo il caricamento, i calendari vengono inseriti nel Picker:

```
CalendarioPicker.ItemsSource = calendari;  
CalendarioPicker.ItemDisplayBinding = new  
Binding(nameof(Calendario.Nome));
```

In questo modo l'utente può selezionare dal menu il calendario che vuole visualizzare.

6.6.9 Memorizzazione dell'ultimo calendario selezionato

Per ricordare l'ultimo calendario usato, viene utilizzato Preferences.

```
int ultimoId = Preferences.Get("CalendarioCorrente", 0);  
calendarioCorrente = calendari.FirstOrDefault(c => c.Id == ultimoId)  
?? calendari.FirstOrDefault();
```

Il programma prova a recuperare l'id dell'ultimo calendario aperto. Se lo trova, imposta quello come calendario corrente. Se invece non esiste più o non è disponibile, viene selezionato il primo calendario della lista.

Quando viene selezionato un calendario, l'id viene salvato:

```
Preferences.Set("CalendarioCorrente", calendarioCorrente.Id);
```

Questo permette all'app di ricordare la scelta dell'utente anche dopo la chiusura.

6.6.10 Caricamento delle attività del calendario

Dopo aver scelto il calendario corrente, vengono caricate le attività associate a quel calendario.



```
async Task CaricaAttivitaCalendario()
{
    if (calendarioCorrente == null) return;

    var attivitaDb = await
database.GetAttivitaCalendario(calendarioCorrente.Id);

    tutteLeAttivita.Clear();
    EventiCalendario.Clear();

    foreach (var aDb in attivitaDb)
    {
        Tipo? tipo = aDb.TipoId.HasValue
            ? tuttiTipi.FirstOrDefault(t => t.Id ==
aDb.TipoId.Value)
            : null;

        var attivita = new Attivita
        {
            Id = aDb.Id,
            Titolo = aDb.Titolo,
            Data = aDb.Data,
            DataFine = aDb.DataFine,
            Colore = Color.FromArgb(aDb.ColoreHex),
            Tipo = tipo,
            Note = aDb.Note,
            Username = aDb.Username,
            CalendarioId = aDb.CalendarioId,
            Completata = aDb.Completata
        };

        tutteLeAttivita.Add(attivita);
        AggiungiACalendario(attivita);
    }

    AggiornaAttivita(DataSelezionata);
}
```

Le attività vengono lette dal database come oggetti AttivitaDb. Successivamente vengono convertite in oggetti Attivita, che sono più adatti alla visualizzazione nella pagina. Questa conversione è necessaria perché il database usa dati semplici, come stringhe e numeri, mentre l'interfaccia grafica usa anche oggetti più complessi, come Color.

6.6.11 Inserimento degli eventi nel calendario grafico

Per mostrare le attività nel calendario, viene usato il metodo AggiungiACalendario.



```
void AggiungiACalendario(Attivita a)
{
    DateTime fine = a.DataFine ?? a.Data;

    for (DateTime d = a.Data.Date; d <= fine.Date; d =
d.AddDays(1))
    {
        if (EventiCalendario.ContainsKey(d))
        {
            var lista =
EventiCalendario[d].Cast<object>().ToList();
            lista.Add(a);
            EventiCalendario[d] = lista;
        }
        else
        {
            EventiCalendario.Add(d, new List<object> { a });
        }
    }
}
```

Questo metodo serve per aggiungere l'attività alla struttura usata dal calendario. Se l'attività dura un solo giorno, viene aggiunta solo a quella data. Se invece possiede una data di fine, viene aggiunta a tutti i giorni compresi tra la data di inizio e la data di fine.

Questa soluzione permette di gestire anche attività lunghe più giorni, come viaggi, progetti o periodi di studio.

6.6.12 Visualizzazione degli eventi del giorno

La lista degli eventi del giorno viene aggiornata con il metodo `AggiornaAttivita`.

```
void AggiornaAttivita(DateTime giorno)
{
    AttivitaDelGiorno.Clear();

    foreach (var a in tutteLeAttivita.OrderBy(a => a.Data))
    {
        DateTime fine = a.DataFine ?? a.Data;

        if (giorno.Date >= a.Data.Date && giorno.Date <= fine.Date)
            AttivitaDelGiorno.Add(a);
    }
}
```

Il metodo svuota prima la lista `AttivitaDelGiorno`, poi controlla tutte le attività presenti. Se la data selezionata rientra tra la data di inizio e la data di fine dell'attività, questa viene aggiunta alla lista.

Nel file XAML questa lista viene mostrata tramite una `CollectionView`:



```
<CollectionView
    ItemsSource="{Binding AttivitaDelGiorno}"
    SelectionMode="Single"
    SelectionChanged="OnAttivitaSelezionata">
</CollectionView>
```

Ogni attività viene visualizzata come una card colorata, con il titolo e due pulsanti: uno per completarla e uno per eliminarla.

6.6.13 Apertura dell'overlay per aggiungere un evento

Per aggiungere un nuovo evento viene usato un overlay, cioè una finestra che appare sopra la pagina principale.

Quando l'utente preme il pulsante "+ Aggiungi evento", viene eseguito questo metodo:

```
void ApriOverlay(object sender, EventArgs e)
{
    if (calendarioCorrente == null)
    {
        DisplayAlert("Errore", "Crea o seleziona prima un
calendario", "OK");
        return;
    }

    DataFinePicker.Date = DataSelezionata.Date;
    Overlay.IsVisible = true;
}
```

Prima viene controllato che esista un calendario selezionato. Se non esiste, viene mostrato un messaggio di errore. Se invece il calendario è presente, l'overlay viene reso visibile. Nel file XAML l'overlay contiene i campi necessari per creare l'evento:



```
<Entry
  x:Name="TitoloEntry"
  Placeholder="Titolo attività"/>

<Picker
  x:Name="ColorePicker"
  Title="Colore"/>

<DatePicker
  x:Name="DataFinePicker"/>

<TimePicker
  x:Name="OraInizioPicker"
  Time="12:00"/>

<TimePicker
  x:Name="OraFinePicker"
  Time="13:00"/>

<Picker
  x:Name="TipoPicker"
  Title="Tipo"
  ItemsSource="{Binding tuttiTipi}"
  ItemDisplayBinding="{Binding Nome}"/>

<Editor
  x:Name="NoteEditor"
  Placeholder="Note dell'attività"/>
```

Questi elementi permettono all'utente di inserire titolo, colore, data di fine, orario, tipo e note.

6.6.14 Aggiunta di una nuova attività

Quando l'utente conferma l'inserimento, viene eseguito il metodo `AggiungiAttivita`.



```
async void AggiungiAttivita(object sender, EventArgs e)
{
    if (calendarioCorrente == null)
    {
        await DisplayAlert("Errore", "Seleziona un calendario", "OK");
        return;
    }

    if (string.IsNullOrWhiteSpace(TitoloEntry.Text))
    {
        await DisplayAlert("Errore", "Inserire un titolo per l'attività", "OK");
        return;
    }

    var tipoSelezionato = TipoPicker.SelectedItem as Tipo;
    if (tipoSelezionato == null)
    {
        await DisplayAlert("Errore", "Scegliere un tipo per l'attività", "OK");
        return;
    }
}
```

Prima del salvataggio vengono fatti diversi controlli. Il programma verifica che sia selezionato un calendario, che il titolo non sia vuoto e che sia stato scelto un tipo di attività.

Dopo i controlli, viene scelto il colore:

```
Color colore = ColorePicker.SelectedIndex switch
{
    0 => Colors.Red,
    1 => Colors.Green,
    2 => Colors.Blue,
    _ => tipoSelezionato.Colore
};
```

Se l'utente sceglie un colore manualmente, viene usato quello. Altrimenti viene usato il colore del tipo selezionato.

Poi vengono costruite la data di inizio e la data di fine:

```
DateTime dataInizio = DataSelezionata.Date +
    OraInizioPicker.Time;

DateTime? dataFine = DataFineSwitch.IsToggled
    ? DataFinePicker.Date + OraFinePicker.Time
    : dataInizio.Date + OraFinePicker.Time;
```

La data di inizio è formata dal giorno selezionato più l'ora scelta dall'utente. La data di fine può essere nello stesso giorno oppure in un altro giorno, se lo switch della data di fine è attivo.

Il programma controlla anche che la data di fine non sia precedente alla data di inizio:



```
if (dataFine.HasValue && dataFine.Value < dataInizio)
{
    await DisplayAlert("Errore", "La data di fine non
    può essere prima di quella d'inizio", "OK");
    return;
}
```

Infine l'attività viene salvata nel database:

```
var attivitaDb = new AttivitaDb
{
    Username = user.Username,
    Titolo = TitoloEntry.Text.Trim(),
    Data = dataInizio,
    DataFine = dataFine,
    ColoreHex = colore.ToArgbHex(),
    TipoId = tipoSelezionato.Id,
    Note = NoteEditor.Text?.Trim() ?? "",
    NotificheAttive = true,
    MinutiPreavviso = 0,
    CalendarioId = calendarioCorrente.Id,
    Completata = false,
    NotificaInviata = false
};

int newId = await database.SalvaAttivita(attivitaDb);
```

Dopo il salvataggio, viene creato anche l'oggetto Attivita da mostrare subito nell'interfaccia:

```
var nuova = new Attivita
{
    Id = newId,
    Titolo = attivitaDb.Titolo,
    Data = dataInizio,
    DataFine = dataFine,
    Colore = colore,
    Tipo = tipoSelezionato,
    Note = attivitaDb.Note,
    Username = user.Username,
    CalendarioId = calendarioCorrente.Id,
    Completata = false
};

tutteLeAttivita.Add(nuova);
AggiungiACalendario(nuova);
AggiornaAttivita(DataSelezionata);
```

Così l'attività viene salvata nel database e compare immediatamente nel calendario.



6.6.15 Eliminazione attività

Ogni attività mostrata nella lista possiede un pulsante per eliminarla.

```
<Button
    Text="X"
    Clicked="EliminaAttivita"
    CommandParameter="{Binding .}"/>
```

Il CommandParameter passa al metodo C# l'attività selezionata.

Il metodo di eliminazione è questo:

```
async void EliminaAttivita(object sender, EventArgs
e)
{
    var attivita = (sender as
Button)?.CommandParameter as Attivita;
    if (attivita == null) return;

    bool conferma = await DisplayAlert(
        "Conferma",
        "Vuoi eliminare questa attività?",
        "Sì",
        "No");

    if (!conferma) return;

    await database.EliminaAttivita(attivita.Id);
    tutteLeAttivita.Remove(attivita);
    RimuoviDaCalendario(attivita);
    AggiornaAttivita(DataSelezionata);
}
```

Prima viene chiesta conferma all'utente. Se conferma, l'attività viene eliminata dal database, rimossa dalla lista locale e tolta anche dal calendario grafico.

6.6.16 Completamento attività

L'utente può segnare un'attività come completata tramite il pulsante con il simbolo di spunta.

```
<Button
    Text="✓"
    Clicked="CompletaAttivita"
    CommandParameter="{Binding .}"/>
```

Il metodo collegato è il seguente:



```
async void CompletaAttivita(object sender,
EventArgs e)
{
    var attivita = (sender as
Button)?.CommandParameter as Attivita;
    if (attivita == null) return;

    attivita.Completata = !attivita.Completata;
    await database.ImpostaCompletata(attivita.Id,
attivita.Completata);
    AggiornaAttivita(DataSelezionata);
}
```

Il valore booleano Completata viene invertito. Se prima era false, diventa true; se era true, torna false. Dopo la modifica, il nuovo stato viene salvato nel database.

6.6.17 Ricerca delle attività

La ricerca permette all'utente di trovare rapidamente un'attività tramite il titolo. La barra di ricerca è definita nel file XAML così:

```
<SearchBar
    x:Name="SearchBarAttivita"
    Placeholder="Cerca attività..."
    SearchButtonPressed="OnSearchPressed"/>
```

Quando l'utente preme il pulsante di ricerca, viene eseguito questo metodo:

```
void OnSearchPressed(object sender, EventArgs
e)
{
    string testo =
SearchBarAttivita.Text?.ToLower().Trim() ?? "";
    RisultatiRicerca.Clear();

    var risultati = tutteLeAttivita
        .Where(a =>
string.IsNullOrEmpty(testo) ||
a.Titolo.ToLower().Contains(testo))
        .OrderBy(a => a.Data)
        .ToList();

    foreach (var a in risultati)
        RisultatiRicerca.Add(a);

    NessunRisultato = risultati.Count == 0;
    SearchOverlay.IsVisible = true;
}
```

Il testo scritto dall'utente viene convertito in minuscolo e confrontato con il titolo delle attività. I risultati vengono inseriti nella collezione RisultatiRicerca e mostrati in un overlay.



Se l'utente seleziona un risultato, il calendario si sposta automaticamente sulla data dell'attività:

```
void OnRisultatoSelezionato(object sender,
SelectionChangedEventArgs e)
{
    var attivita =
e.CurrentSelection.FirstOrDefault() as
Attivita;
    if (attivita == null) return;

    DataSelezionata = attivita.Data.Date;
    SearchOverlay.IsVisible = false;
}
```

6.6.18 Visualizzazione del dettaglio di un'attività

Quando l'utente clicca su un'attività, viene aperto un overlay con i dettagli completi.

```
void OnAttivitaSelezionata(object sender,
SelectionChangedEventArgs e)
{
    var attivita = e.CurrentSelection.FirstOrDefault() as
Attivita;
    if (attivita == null) return;

    DettaglioHeader.BackgroundColor = attivita.Colore;
    DettaglioTitolo.Text = attivita.Titolo;
    DettaglioDataInizio.Text = attivita.Data.ToString("dd MMMM
yyyy HH:mm");

    DettaglioDataFine.Text = attivita.DataFine.HasValue
        ? attivita.DataFine.Value.ToString("dd MMMM yyyy HH:mm")
        : "Nessuna";

    DettaglioNote.Text = string.IsNullOrWhiteSpace(attivita.Note)
        ? "Nessuna nota"
        : attivita.Note;

    OverlayDettaglio.IsVisible = true;
}
```

Questo overlay mostra titolo, data di inizio, data di fine, colore, tipo e note. In questo modo l'utente può vedere tutte le informazioni dell'attività senza dover aprire un'altra pagina.

6.6.19 Creazione di un nuovo calendario

L'utente può creare un nuovo calendario tramite il pulsante "+". Quando viene premuto, viene richiesto il nome del calendario.



```
async void CreaCalendarioClicked(object sender, EventArgs e)
{
    string nome = await DisplayPromptAsync("Nuovo calendario",
    "Nome calendario");
    if (string.IsNullOrEmpty(nome)) return;

    var user = UserSession.CurrentUser;
    if (user == null) return;

    var calendario = new Calendario
    {
        Nome = nome.Trim(),
        CreatoreUsername = user.Username,
        Condiviso = false
    };

    int calendarioId = await
    database.SalvaCalendario(calendario);

    await database.AggiungiUtenteCalendario(new CalendarioUtente
    {
        CalendarioId = calendarioId,
        Username = user.Username
    });

    Preferences.Set("CalendarioCorrente", calendarioId);
    await CaricaCalendari();
}
```

Il calendario viene salvato nella tabella Calendario. Subito dopo viene creata anche una relazione nella tabella CalendarioUtente, che collega l'utente al calendario appena creato.

6.6.20 Cambio del calendario selezionato

Quando l'utente seleziona un calendario diverso dal Picker, viene eseguito il metodo CalendarioCambiato.

```
async void CalendarioCambiato(object sender, EventArgs e)
{
    calendarioCorrente = CalendarioPicker.SelectedItem as
    Calendario;
    if (calendarioCorrente == null) return;

    Preferences.Set("CalendarioCorrente",
    calendarioCorrente.Id);
    await CaricaAttivitaCalendario();
}
```

Il programma aggiorna il calendario corrente, salva la scelta nelle preferenze e ricarica tutte le attività collegate al nuovo calendario selezionato.



6.6.21 Condivisione del calendario con altri utenti

Una funzione importante è la possibilità di invitare altri utenti in un calendario. L'utente inserisce lo username della persona da invitare.

```
async void InvitaClicked(object sender, EventArgs e)
{
    if (calendarioCorrente == null) return;

    string username = await DisplayPromptAsync("Invita
utente", "Username");
    if (string.IsNullOrEmpty(username)) return;

    username = username.Trim();
    var user = UserSession.CurrentUser;
    if (user == null) return;
}
```

Prima di inviare l'invito vengono effettuati diversi controlli. Il programma impedisce di invitare sé stessi e verificare che l'utente esista. Inoltre, controlla se l'utente non sia già dentro al calendario e se esiste già una richiesta pendente. Se tutti i controlli vengono superati, viene creata la richiesta:

```
await database.InviaRichiesta(new RichiestaCondivisione
{
    MittenteUsername = user.Username,
    DestinatararioUsername = username,
    CalendarioId = calendarioCorrente.Id,
    Stato = "In attesa"
});
```

6.6.22 Gestione richieste ricevute

Le richieste ricevute vengono mostrate tramite il pulsante con l'icona della campana. Quando l'utente lo preme, vengono caricate tutte le richieste pendenti.

```
async void MostraRichiesteClicked(object sender, EventArgs e)
{
    var user = UserSession.CurrentUser;
    if (user == null) return;

    var richieste = await database.GetRichieste(user.Username);

    if (richieste.Count == 0)
    {
        await DisplayAlert("Richieste", "Nessuna richiesta
pendente", "OK");
        return;
    }
}
```

Per ogni richiesta viene mostrata una scelta tra "Accetta" e "Rifiuta". Se l'utente accetta, viene aggiunto al calendario.



6.6.23 Database e salvataggio dei dati

Il salvataggio dei dati viene gestito dal servizio DatabaseService. All'avvio vengono create le tabelle necessarie:

```
await database.CreateTableAsync<User>();  
await database.CreateTableAsync<AttivitaDb>();  
await database.CreateTableAsync<TipoDb>();  
await database.CreateTableAsync<Calendario>();  
await database.CreateTableAsync<CalendarioUtente>();  
await database.CreateTableAsync<RichiestaCondivisione>();
```

6.6.24 Gestione del tema grafico

La pagina calendario supporta anche il tema scelto dall'utente. Quando la pagina viene aperta, viene letto il valore salvato nelle preferenze.

```
void LoadTheme()  
{  
    int index = Preferences.Get("AppTheme", 0);  
  
    switch (index)  
    {  
        case 1: ThemeManager.SetDarkTheme(); break;  
        case 2: ThemeManager.SetBlueTheme(); break;  
        case 3: ThemeManager.SetPinkTheme(); break;  
        case 4: ThemeManager.SetPurpleTheme(); break;  
        case 5: ThemeManager.SetYellowTheme(); break;  
        case 6: ThemeManager.SetRedTheme(); break;  
        case 7: ThemeManager.SetBrownTheme(); break;  
        default: ThemeManager.SetLightTheme(); break;  
    }  
  
    ApplyTheme();  
}
```

Poi il tema viene applicato agli elementi principali:

```
void ApplyTheme()  
{  
    BackgroundColor = ThemeManager.BackgroundColor;  
    LabelNomeUtente.TextColor = ThemeManager.TextColor;  
}
```

Questa funzione rende il calendario coerente con il resto dell'applicazione.

6.7 Statistiche, Profilo, Modifica Profilo e Logout

Oltre alla pagina calendario, l'applicazione OraX contiene altre sezioni importanti per l'utente: la pagina delle statistiche, la pagina del profilo, la pagina di modifica del profilo e la funzione di logout. Queste parti servono per migliorare l'esperienza d'uso dell'applicazione, permettendo all'utente di controllare l'andamento delle proprie attività, visualizzare i propri dati personali, modificarli e uscire dal proprio account.



Dal punto di vista tecnico, queste pagine sono state realizzate sempre con .NET MAUI, utilizzando file XAML per la parte grafica e file C# per la parte logica.

6.7.1 Statistiche

La pagina delle statistiche è contenuta nei file:

- Statistiche.xaml
- Statistiche.xaml.cs

Questa pagina ha lo scopo di mostrare all'utente un riassunto delle sue attività. In particolare vengono visualizzati il numero di attività completate, quelle non completate, il totale delle attività e la percentuale di completamento.

Nel file XAML la pagina è organizzata con una ScrollView e una VerticalStackLayout, in modo che tutti gli elementi siano disposti verticalmente e siano leggibili anche su schermi più piccoli.

6.7.1.1 Caricamento

La logica principale si trova nel metodo CaricaStatistiche.

```
async Task CaricaStatistiche()
{
    user = UserSession.CurrentUser;

    if (user == null)
    {
        labelFatte.Text = "0";
        labelNonFatte.Text = "0";
        labelTot.Text = "Totale: 0 attività";
        labelPercento.Text = "0% completato";
        progressBar.Progress = 0;
        return;
    }

    int calendarioCorrente =
        Preferences.Get("CalendarioCorrente", 0);

    var attivita = calendarioCorrente > 0
        ? await
        database.GetAttivitaUtenteCalendario(calendarioCorrente,
        user.Username)
        : await database.GetAttivitaUtente(user.Username);
}
```

Per prima cosa viene recuperato l'utente attualmente collegato tramite UserSession.CurrentUser. Se non è presente nessun utente, la pagina mostra tutti i valori a zero.

Successivamente viene letto dalle Preferences il calendario corrente. Se esiste un calendario selezionato, vengono caricate solo le attività di quel calendario. Altrimenti vengono caricate tutte le attività dell'utente.



6.7.1.2 Calcolo dei dati

Dopo aver caricato le attività, il programma calcola quante sono completate, quante non sono completate e il totale.

```
int complete = attivita.Count(a => a.Completata);
int nonCompleate = attivita.Count(a => !a.Completata);
int totale = attivita.Count;

double percentuale = totale > 0
    ? (double)complete / totale * 100
    : 0;
```

La proprietà Completata è un valore booleano. Se vale true, l'attività viene considerata completata; se vale false, viene considerata non completata.

La percentuale viene calcolata dividendo il numero di attività completate per il totale delle attività. Il controllo totale > 0 evita errori nel caso in cui l'utente non abbia ancora nessuna attività.

6.7.1.3 Attività recenti

La pagina mostra anche un piccolo riassunto delle attività recenti: oggi, ieri e due giorni fa.

```
AggiornaRigaGiorno(attivita, DateTime.Today, labelQuante);
AggiornaRigaGiorno(attivita, DateTime.Today.AddDays(-1),
    labelStatistiche);
AggiornaRigaGiorno(attivita, DateTime.Today.AddDays(-2),
    labelStati);
```

Il metodo usato è questo:

```
void AggiornaRigaGiorno(IEnumerable<AttivitaDb> attivita,
    DateTime giorno, Label label)
{
    var delGiorno = attivita.Where(a => a.Data.Date ==
        giorno.Date);

    int complete = delGiorno.Count(a => a.Completata);
    int nonComplete = delGiorno.Count(a => !a.Completata);

    label.Text = $"{complete} completate · {nonComplete} non
        completate";
}
```

Questo metodo riceve una data e controlla quante attività appartengono a quel giorno. Poi aggiorna la label corrispondente.

6.7.2 Profilo

La pagina profilo corrisponde alla MainPage, contenuta nei file:

- MainPage.xaml



- MainPage.xaml.cs

Questa pagina mostra i dati principali dell'utente collegato, cioè nome, cognome, email e immagine del profilo. Inoltre contiene i pulsanti per accedere al calendario, alle statistiche, alla modifica del profilo e al logout.

6.7.2.1 Caricamento dati

```
public MainPage()
{
    InitializeComponent();
    user = UserSession.CurrentUser;

    if (user != null)
    {
        labelNome.Text = $"{user.Nome} {user.Cognome}";
        labelEmail.Text = user.Email;

        if (!string.IsNullOrEmpty(user.FotoProfiloPath))
            ImgProfilo.Source = user.FotoProfiloPath;
    }

    LoadTheme();
    ThemeManager.ThemeChanged += ApplyTheme;
}
```

Se l'utente esiste, il programma mostra nome, cognome ed email. Se l'utente ha già scelto una foto profilo, viene caricata dal percorso salvato in FotoProfiloPath.

6.7.2.2 Navigazione

La pagina contiene diversi bottoni che permettono di navigare e accedere alla pagina delle statistiche, delle modifiche, del calendario e per disconnettersi dall'account.

6.7.2.3 Selezione del tema

Nella pagina profilo è presente anche un Picker che permette di scegliere il tema dell'applicazione. Quando l'utente cambia tema, viene eseguito il metodo OnThemeChanged.



```
private void OnThemeChanged(object sender, EventArgs e)
{
    var picker = sender as Picker;
    int index = picker.SelectedIndex;

    switch (index)
    {
        case 0: ThemeManager.SetLightTheme(); break;
        case 1: ThemeManager.SetDarkTheme(); break;
        case 2: ThemeManager.SetBlueTheme(); break;
        case 3: ThemeManager.SetPinkTheme(); break;
        case 4: ThemeManager.SetPurpleTheme(); break;
        case 5: ThemeManager.SetYellowTheme(); break;
        case 6: ThemeManager.SetRedTheme(); break;
        case 7: ThemeManager.SetBrownTheme(); break;
        default: ThemeManager.SetLightTheme(); break;
    }

    Preferences.Set("AppTheme", index);
    ApplyTheme();
}
```

La scelta viene salvata nelle Preferences, così il tema rimane anche quando l'app viene chiusa e riaperta.

6.7.3 Modifica profilo

La pagina di modifica profilo è contenuta nei file:

- Modifiche.xaml
- Modifiche.xaml.cs

Questa pagina permette all'utente di cambiare i propri dati personali, come nome, cognome, email, telefono e foto profilo.

6.7.3.1 Caricamento dati

Nel costruttore viene recuperato l'utente corrente e i suoi dati vengono inseriti nei campi della pagina.



```
public Modifiche()
{
    InitializeComponent();

    database = new DatabaseService();
    user = UserSession.CurrentUser;

    if (user != null)
    {
        NameEntry.Text = user.Nome;
        SurnameEntry.Text = user.Cognome;
        UsernameEntry.Text = user.Username;
        EmailEntry.Text = user.Email;
        PhoneEntry.Text = user.Telefono;

        if (!string.IsNullOrEmpty(user.FotoProfiloPath))
        {
            ImgProfilo.Source = user.FotoProfiloPath;
        }
    }
}
```

In questo modo, quando l'utente apre la pagina, vede già i propri dati attuali e può modificarli direttamente.

6.7.3.2 Foto profilo

Per cambiare la foto profilo viene usato il FilePicker.

```
private async void OnChangePhotoClicked(object sender, EventArgs e)
{
    var result = await FilePicker.PickAsync(new PickOptions
    {
        PickerTitle = "Scegli una foto",
        FileTypes = FilePickerFileType.Images
    });

    if (result == null)
        return;
}
```

Il programma apre il selettore dei file e permette di scegliere solo immagini. Se l'utente non sceglie nulla, il metodo termina.

Dopo la scelta, l'immagine viene copiata nella cartella interna dell'applicazione.



```
string localPath = Path.Combine(FileSystem.AppDataDirectory,
result.FileName);

using (var stream = await result.OpenReadAsync())
using (var newStream = File.OpenWrite(localPath))
{
    await stream.CopyToAsync(newStream);
}
```

Infine viene aggiornato il percorso della foto profilo e l'immagine viene mostrata subito nella pagina.

```
user.FotoProfiloPath = localPath;
ImgProfilo.Source = localPath;
```

Questa soluzione è utile perché salva una copia locale dell'immagine, evitando di dipendere dal file originale scelto dall'utente.

6.7.4 Disconnessione

Il logout si trova nella pagina profilo. Nel file XAML è presente il pulsante:

```
<Button Text="Esci"
        Clicked="OnLogoutClicked"
        BackgroundColor="Transparent"
        TextColor="#FF3B30"/>
```

Quando l'utente preme "Esci", viene mostrata una finestra di conferma.

```
private async void OnLogoutClicked(object sender,
EventArgs e)
{
    bool conferma = await DisplayAlert(
        "Logout",
        "Sei sicuro di voler uscire?",
        "S",
        "No"
    );

    if (!conferma)
        return;
}
```

Se l'utente sceglie "No", il metodo si interrompe e non succede nulla. Questo evita uscite accidentali.

6.7.4.1 Pulizia dei dati

Se l'utente conferma, vengono rimossi alcuni dati salvati nelle Preferences.



Questi dati servono per ricordare l'utente collegato. Rimuovendoli, l'app non mantiene più la sessione precedente.

```
Preferences.Remove("utente_id");  
Preferences.Remove("email");  
Preferences.Remove("nome");  
Preferences.Remove("ruolo");
```

Dopo aver cancellato i dati, l'utente viene riportato alla pagina di login.

6.8 Notifiche

Per permettere all'utente di ricevere promemoria relativi alle attività create, è stato implementato un sistema di notifiche basato sul servizio NotificationService. Questo componente controlla periodicamente le attività salvate nel database e verifica se una di esse è arrivata alla data e ora prevista.

```
public class NotificationService  
{  
    private readonly DatabaseService  
    database;  
    private IDispatcherTimer? timer;  
}
```

Le notifiche vengono controllate tramite un timer che esegue una verifica ogni 30 secondi.

```
timer.Interval = TimeSpan.FromSeconds(30);  
  
timer.Tick += async (s, e) =>  
{  
    await ControllaNotifiche();  
};
```

Questo permette all'applicazione di controllare costantemente se esistono attività in scadenza.

6.8.1.1 Gestione delle attività notificabili

Nel database ogni attività contiene alcune informazioni dedicate alle notifiche.

- public bool NotificheAttive { get; set; }
- public bool NotificaInviata { get; set; }

Questi campi permettono di sapere se una notifica deve essere mostrata e di evitare che la stessa attività venga notificata più volte.

6.8.1.2 Visualizzazione notifica

Quando viene rilevata un'attività in scadenza, l'applicazione mostra un messaggio all'utente.

```
await pagina.DisplayAlert(  
    "Promemoria",  
    $"L'attività \"{item.Titolo}\" scade  
    adesso.",  
    "OK");
```

L'utente riceve quindi un avviso immediato contenente il nome dell'attività.



6.9 Conclusione

L'implementazione del progetto OraX ha permesso di trasformare le funzionalità progettate nella fase di analisi in un'applicazione completa, funzionante e utilizzabile dagli utenti. Durante lo sviluppo sono state realizzate tutte le componenti principali del sistema, integrando interfacce grafiche moderne con una gestione efficace dei dati tramite database SQLite.

Particolare attenzione è stata dedicata alla realizzazione del calendario, che rappresenta il cuore dell'applicazione. Grazie ad esso l'utente può creare, modificare e gestire le proprie attività in modo semplice e intuitivo, organizzandole all'interno di calendari personali o condivisi. Sono state inoltre implementate funzionalità aggiuntive come la ricerca delle attività, la visualizzazione dei dettagli, il completamento delle attività e il sistema di condivisione tra utenti.

La pagina delle statistiche consente invece di monitorare l'andamento delle attività e di visualizzare informazioni utili sul livello di completamento degli impegni. Le pagine dedicate al profilo e alla modifica del profilo permettono all'utente di personalizzare la propria esperienza d'uso aggiornando i dati personali e l'immagine del profilo.

Per migliorare ulteriormente l'organizzazione delle attività è stato sviluppato anche un sistema di notifiche che controlla automaticamente le scadenze e avvisa l'utente quando un'attività richiede attenzione. Inoltre, la presenza di temi personalizzabili permette di adattare l'aspetto grafico dell'applicazione alle preferenze di ogni utente.

Dal punto di vista tecnico, il progetto è stato sviluppato utilizzando .NET MAUI, adottando una chiara separazione tra interfaccia grafica, logica applicativa e gestione dei dati. Questa scelta ha reso il codice più ordinato, manutenibile ed espandibile nel tempo. L'utilizzo di SQLite ha garantito una gestione affidabile delle informazioni, consentendo il salvataggio permanente dei dati anche dopo la chiusura dell'applicazione.

In conclusione, il risultato ottenuto soddisfa gli obiettivi inizialmente prefissati, offrendo uno strumento completo per la gestione delle attività personali e condivise. Il progetto ha inoltre permesso di approfondire numerose competenze tecniche, tra cui lo sviluppo di applicazioni multiplatforma, la gestione dei database, la progettazione delle interfacce utente e l'organizzazione del codice secondo una struttura professionale.



7 Test

7.1 Protocollo di test

Il seguente capitolo si occupa dei test, coloro che permettono di verificare il corretto funzionamento del programma. Ogni azione che l'utente andrà a fare bisognerà testarlo.

Test Case:	TC-001	Nome:	Registrazione nuovo utente
Riferimento:	REQ-01		
Descrizione:	Verificare che un utente possa registrarsi correttamente nel sistema.		
Prerequisiti:	• Applicazione avviata • Nessun account con lo stesso username		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di registrazione; 3. Inserire dati validi; 4. Premere "Registrati";		
Risultati attesi:	L'account viene creato e registrato all'interno del database.		

Test Case:	TC-002	Nome:	Accesso al sistema
Riferimento:	REQ-02		
Descrizione:	Verificare che un utente possa accedere correttamente al sistema.		
Prerequisiti:	• Applicazione avviata • Avere un account		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi";		
Risultati attesi:	L'account viene riconosciuto e l'utente accede al suo calendario.		

Test Case:	TC-003	Nome:	Visualizzazione calendario
Riferimento:	REQ-03		
Descrizione:	Verificare che il calendario venga visualizzato dopo il login.		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario;		



Risultati attesi:	L'utente riesce a visualizzare il calendario.
--------------------------	---

Test Case:	TC-004	Nome:	Inserire attività
Riferimento:	REQ-04		
Descrizione:	Verificare che l'utente possa inserire un'attività all'interno del suo calendario.		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario; 6. Inserire un'attività;		
Risultati attesi:	L'utente riesce ad inserire un'attività all'interno del suo calendario.		

Test Case:	TC-005	Nome:	Inserire dettagli attività
Riferimento:	REQ-05		
Descrizione:	Verificare che l'utente possa inserire i dettagli di un'attività quando la inserisce.		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario • Inserire un'attività		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario; 6. Inserire un'attività; 7. Specificare i dettagli;		
Risultati attesi:	L'utente riesce ad inserire i dettagli di un'attività.		

Test Case:	TC-006	Nome:	Modificare attività
Riferimento:	REQ-06		
Descrizione:	Verificare che l'utente possa modificare un'attività.		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario		



	• Avere un'attività
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario; 6. Premere un'attività; 7. Modificarla;
Risultati attesi:	L'utente riesce a modificare un'attività.

Test Case:	TC-007	Nome:	Eliminare attività
Riferimento:	REQ-06		
Descrizione:	Verificare che l'utente possa eliminare un'attività.		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario • Avere un'attività		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario; 6. Premere un'attività; 7. Eliminarla;		
Risultati attesi:	L'utente riesce ad eliminare un'attività.		

Test Case:	TC-008	Nome:	Visualizzare profilo
Riferimento:	REQ-02		
Descrizione:	Verificare che l'utente possa visualizzare il suo profilo;		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario		
Procedura:	8. Avviare l'applicativo; 9. Accedere alla pagina di login; 10. Inserire credenziali; 11. Premere "Accedi"; 12. Visualizzare il calendario; 13. Tramite la barra di navigazione, accedere al profilo;		



Risultati attesi:	L'utente riesce a vedere il suo account corretto.
--------------------------	---

Test Case:	TC-009	Nome:	Modifica profilo
Riferimento:	REQ-02		
Descrizione:	Verificare che l'utente possa modificare il suo profilo;		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario		
Procedura:	14. Avviare l'applicativo; 15. Accedere alla pagina di login; 16. Inserire credenziali; 17. Premere "Accedi"; 18. Visualizzare il calendario; 19. Tramite la barra di navigazione, accedere alle modifiche; 20. Modificare i dati; 21. Salvare;		
Risultati attesi:	L'utente riesce a modificare il suo profilo e a salvarlo.		

Test Case:	TC-010	Nome:	Personalizzazione calendario
Riferimento:	REQ-08		
Descrizione:	Verificare che l'utente possa personalizzare il tema del suo calendario.		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario; 6. Accedere al profilo; 7. Cambiare tema;		
Risultati attesi:	L'utente riesce a cambiare il tema del calendario a dipendenza delle sue preferenze.		

Test Case:	TC-011	Nome:	Statistiche
Riferimento:	REQ-09		
Descrizione:	Verificare che l'utente visualizzi le statistiche delle sue attività.		
Prerequisiti:	• Applicazione avviata		



	• Accedere con le proprie credenziali • Visualizzare il calendario
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario; 6. Accedere alla pagina profilo; 7. Premere il bottone "Visualizza statistiche";
Risultati attesi:	L'utente riesce a visualizzare le statistiche inerenti alle sue attività e sono corrette.

Test Case:	TC-012	Nome:	Ricerca delle attività
Riferimento:	REQ-11		
Descrizione:	Verificare che l'utente possa cercare le attività.		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario • Avere un'attività		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario; 6. Cercare l'attività nella barra di ricerca;		
Risultati attesi:	L'utente riesce a cercare l'attività desiderata.		

Test Case:	TC-013	Nome:	Condivisione del calendario
Riferimento:	REQ-12		
Descrizione:	Verificare che l'utente possa condividere le proprie attività con un altro utente.		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario; 6. Premere "+" per aggiungere un nuovo calendario;		



	7. Inserire nome del calendario; 8. Premere il bottone dell'invito; 9. Inserire username dell'utente con cui si vuole condividere il calendario; 10. Attendere che l'utente accetti la richiesta;
Risultati attesi:	L'utente riesce a condividere con altri utenti le sue attività.

Test Case:	TC-014	Nome:	Richiesta di condivisione
Riferimento:	REQ-13		
Descrizione:	Verificare che l'utente possa ricevere la richiesta di condivisione.		
Prerequisiti:	• Applicazione avviata • Accedere con le proprie credenziali • Visualizzare il calendario		
Procedura:	1. Avviare l'applicativo; 2. Accedere alla pagina di login; 3. Inserire credenziali; 4. Premere "Accedi"; 5. Visualizzare il calendario; 6. Premere la campanella per visualizzare le richieste; 7. Accettare o rifiutare la richiesta di condivisione;		
Risultati attesi:	L'utente riesce a ricevere la notifica di condivisione.		



7.2 Risultati test

Test	Esito
TC-01	Riuscito
TC-02	Riuscito
TC-03	Riuscito
TC-04	Riuscito
TC-05	Riuscito
TC-06	Riuscito
TC-07	Riuscito
TC-08	Riuscito
TC-09	Riuscito
TC-010	Riuscito
TC-011	Riuscito
TC-012	Riuscito
TC-013	Riuscito
TC-014	Riuscito

7.3 Mancanze/limitazioni conosciute

Nonostante il progetto **OraX** sia stato completato nelle sue funzionalità principali, ci sono alcune limitazioni e miglioramenti che non è stato possibile implementare durante il tempo a disposizione per il progetto.

1) Layout uniforme delle pagine

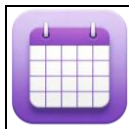
Attualmente l'interfaccia grafica non presenta uno stile completamente uniforme in tutte le schermate dell'applicazione. Alcune pagine come quella di login utilizza dimensioni, spaziature e disposizioni degli elementi leggermente differenti e più moderni. Questa limitazione non compromette il funzionamento dell'applicazione ma potrebbe influire sull'esperienza utente.

2) Contatore delle notifiche non implementato

Il sistema di notifiche consente di ricevere e visualizzare gli avvisi relativi alle attività, tuttavia non è presente un contatore numerico che mostri in tempo reale il numero di notifiche o che semplicemente faccia notare che c'è una notifica non letta. Questa funzionalità migliorerebbe la gestione degli avvisi e l'usabilità in generale, rendendola più intuitiva.

3) Condivisione limitata dei calendari

La condivisione è stata implementata in forma basilare e non include funzionalità avanzate come ruoli differenti per gli utenti condivisi, gestione dei permessi.



SAMT – Sezione Informatica

Implementazione di OraX in .NET MAUI

Pagina 61 di 82

8 Consuntivo

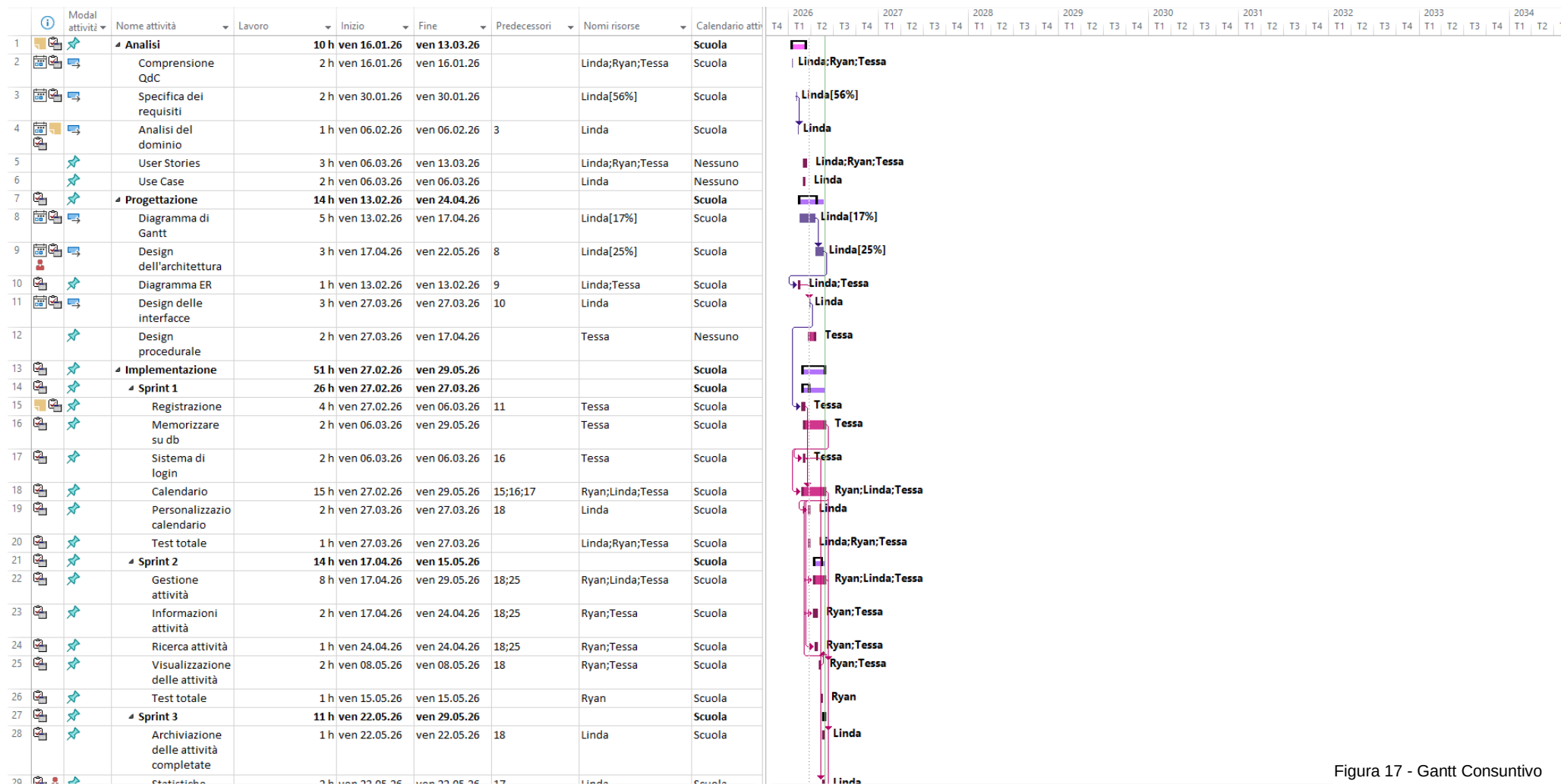


Figura 17 - Gantt Consuntivo



8.1 Differenze – Gantt Preventivo e Consuntivo

Durante la fase iniziale del progetto è stato realizzato un Gantt preventivo, utilizzato per pianificare le attività da svolgere, stimare i tempi necessari e organizzare le varie fasi di sviluppo dell'applicazione OraX. Questo diagramma rappresentava una previsione del lavoro e costituiva una guida per l'avanzamento del progetto.

Al termine dello sviluppo è stato invece realizzato il Gantt consuntivo, che riporta i tempi effettivamente impiegati per ogni attività. Il confronto tra i due diagrammi ha permesso di verificare l'accuratezza della pianificazione iniziale e di individuare eventuali differenze tra quanto previsto e quanto realmente svolto.

In generale, alcune attività hanno richiesto più tempo del previsto, in particolare la realizzazione del calendario, del sistema di condivisione dei calendari e delle notifiche, poiché durante lo sviluppo sono emerse problematiche tecniche non previste nella fase di progettazione. Al contrario, attività come la creazione delle pagine profilo e statistiche hanno richiesto tempi inferiori rispetto a quelli stimati inizialmente.

Nonostante queste variazioni, il progetto è stato completato con successo e tutte le funzionalità principali previste nei requisiti sono state implementate. Il Gantt consuntivo evidenzia quindi l'effettivo andamento del lavoro e rappresenta uno strumento utile per valutare l'organizzazione del progetto e migliorare future pianificazioni.



9 Conclusioni

Il progetto OraX non nasce con l'obiettivo di rivoluzionare il settore delle applicazioni di produttività o di introdurre tecnologie innovative a livello mondiale. Esistono infatti già numerose applicazioni dedicate alla gestione di attività e calendari. Tuttavia, il valore del progetto non risiede nell'invenzione di una nuova categoria di software, bensì nella capacità di progettare e realizzare una soluzione completa, funzionante e adatta alle esigenze degli utenti a cui è destinata.

L'applicazione sviluppata permette di organizzare attività, gestire calendari personali e condivisi, ricevere notifiche e monitorare i propri progressi attraverso statistiche dedicate. Queste funzionalità contribuiscono a migliorare l'organizzazione del tempo e la gestione degli impegni quotidiani, offrendo agli utenti uno strumento pratico e intuitivo.

Dal punto di vista dell'impatto, la soluzione può risultare utile soprattutto in contesti scolastici, personali o collaborativi di piccole dimensioni, dove la semplicità d'uso e la possibilità di condividere calendari rappresentano un vantaggio concreto. Pur non avendo un impatto globale, il progetto dimostra come sia possibile realizzare un'applicazione completa partendo da un'esigenza reale e trasformandola in un prodotto funzionante.

I risultati ottenuti sono in parte specifici del caso studiato, poiché l'applicazione è stata progettata tenendo conto di determinati requisiti e di uno specifico contesto d'utilizzo. Tuttavia, molte delle soluzioni adottate, come la gestione degli utenti, l'utilizzo di database locali, la condivisione delle informazioni e la personalizzazione dell'interfaccia, sono facilmente riutilizzabili e adattabili ad altri progetti software con caratteristiche simili.

Un altro aspetto importante riguarda il percorso di sviluppo stesso. Oltre al prodotto finale, il progetto ha permesso di acquisire esperienza pratica nella progettazione, nell'implementazione e nella gestione di un'applicazione reale. Sono state approfondite competenze relative allo sviluppo multiplatforma con .NET MAUI, alla gestione dei database SQLite, all'organizzazione del codice e al lavoro su un progetto di dimensioni significative.

In conclusione, il progetto non rappresenta una rivoluzione tecnologica né una soluzione destinata a cambiare il mondo, ma costituisce un risultato significativo dal punto di vista formativo e tecnico. L'applicazione raggiunge gli obiettivi prefissati, fornisce una soluzione concreta al problema affrontato e dimostra la capacità di sviluppare un software completo, mantenendo al tempo stesso una base solida per eventuali evoluzioni future.



9.1 Sviluppi futuri

Nonostante il progetto OraX abbia raggiunto gli obiettivi prefissati e presenti tutte le funzionalità principali necessarie per la gestione delle attività, esistono diversi aspetti che potrebbero essere ulteriormente migliorati e ampliati in future versioni dell'applicazione.

Un primo sviluppo riguarda l'interfaccia grafica. Sebbene l'applicazione presenti già uno stile moderno e personalizzabile tramite diversi temi, sarebbe possibile uniformare ulteriormente il layout delle varie pagine, rendendo l'esperienza utente ancora più coerente. Alcune schermate potrebbero infatti essere adattate per utilizzare la stessa disposizione degli elementi, gli stessi spazi e lo stesso stile grafico, così da offrire una maggiore continuità durante la navigazione.

Un secondo miglioramento potrebbe interessare il sistema di condivisione dei calendari. Attualmente gli utenti possono inviare e ricevere richieste di condivisione, ma sarebbe utile introdurre un indicatore numerico associato all'icona delle notifiche. In questo modo l'utente potrebbe visualizzare immediatamente il numero di richieste ancora da leggere o gestire, senza dover aprire ogni volta la schermata dedicata.

Un'ulteriore evoluzione potrebbe riguardare la gestione delle attività nei calendari condivisi. Attualmente è possibile visualizzare le informazioni principali di ogni attività, ma nei dettagli dell'evento potrebbe essere aggiunto anche il nome dell'utente che ha creato l'attività. Questa funzionalità permetterebbe di identificare facilmente il responsabile di ogni evento e renderebbe più efficace la collaborazione tra più utenti all'interno dello stesso calendario.

In conclusione, il progetto OraX rappresenta una solida base sulla quale costruire nuove funzionalità e miglioramenti. Grazie alla struttura modulare adottata durante lo sviluppo, l'applicazione può essere ampliata con relativa facilità, permettendo l'aggiunta di nuove caratteristiche senza modificare in modo significativo quelle già esistenti. Gli sviluppi futuri descritti contribuirebbero a migliorare ulteriormente l'usabilità, l'organizzazione e l'esperienza complessiva degli utenti.

9.2 Considerazioni personali

9.2.1 Linda

Durante lo sviluppo del progetto ho avuto l'opportunità di approfondire numerose competenze tecniche legate allo sviluppo di applicazioni con .NET MAUI, alla gestione di database SQLite e all'organizzazione di un progetto software completo. Dal punto di vista tecnico, il progetto è stato molto utile perché mi ha permesso di mettere in pratica conoscenze apprese durante il percorso scolastico e di affrontare problematiche reali che richiedevano ricerca, sperimentazione e capacità di problem solving.

Dal punto di vista organizzativo, invece, il lavoro di gruppo ha rappresentato una delle maggiori difficoltà incontrate durante il progetto. In alcuni momenti la suddivisione dei compiti non è stata equilibrata e non tutti i membri del gruppo hanno contribuito con lo stesso impegno e la stessa quantità di lavoro. Questo ha portato alcune persone a dover gestire una parte più consistente dello sviluppo, mentre altre si sono occupate di attività meno impegnative o meno numerose.

Un'altra difficoltà è stata il coordinamento tra i membri del gruppo, soprattutto nella gestione delle tempistiche e nell'organizzazione delle attività da svolgere. In alcuni casi è stato necessario rivedere la pianificazione iniziale o completare parti di lavoro che non erano state portate avanti come previsto. Nonostante queste difficoltà, il progetto è stato portato a termine con successo e ha rappresentato un'importante esperienza formativa. Mi ha permesso di comprendere quanto sia importante una corretta distribuzione dei compiti, una comunicazione efficace all'interno del gruppo e una buona organizzazione del lavoro. Sono aspetti che ritengo fondamentali per affrontare in modo più efficiente progetti futuri, sia in ambito scolastico sia professionale.



Nel complesso considero questa esperienza positiva, perché oltre alle competenze tecniche mi ha insegnato anche l'importanza della collaborazione e della gestione delle dinamiche di gruppo, aspetti che fanno parte integrante dello sviluppo di qualsiasi progetto informatico.

9.2.2 Tessa

La realizzazione di questo progetto è stata un'esperienza molto formativa, soprattutto perché era il mio progetto di gruppo. Mi sono sempre trovata veramente bene a lavorare da sola, infatti farlo con altre 2 persone è stato difficile, ha i suoi vantaggi e svantaggi.

Con il tempo mi sono abituata a questo metodo di lavoro e mi sento molto migliorata, questa esperienza mi permetterà di essere preparata in futuro quando dovrò lavorare con dei colleghi.

Attraverso questo lavoro ho avuto l'opportunità di approfondire l'utilizzo di tecnologie come .NET MAUI, C#, XAML e SQLite, che non usavo da un anno, imparando a sviluppare un'applicazione completa partendo dall'analisi fino alla fase finale.

Sono soddisfatta del risultato ottenuto perché il prodotto finale rispecchia gli obiettivi principali definiti all'inizio del progetto e offre tante funzionalità che reputo molto utili, soprattutto per studenti come noi.

Nonostante siano presenti alcuni aspetti migliorabili, il progetto rappresenta per me un importante traguardo e molta esperienza in più, sia dal lato di comunicazione, che nella pratica e nell'organizzazione.

In conclusione questo progetto mi ha fatto acquisire maggiore sicurezza in me stessa e nella programmazione e mi ha dimostrato che, con impegno e costanza, è possibile realizzare applicazioni complesse e funzionali partendo da un'idea iniziale organizzata a dovere, anche con l'aiuto di altri.

9.2.3 Ryan

Personalmente ho capito che è stato molto più difficile di quanto pensassi gestire un progetto con la metodologia Agile con altre due persone. Mi sono ritrovato molto spiazzato nel gestire la pianificazione che del progetto che è stata molto difficile da seguire a causa di problemi col linguaggio e problemi con l'unione delle varie parti del codice.

Poi personalmente penso che ho per adesso perso interesse nello sviluppo di applicazioni di questo tipo e penso che in futuro se sviluppo con la metodologia Waterflow e sviluppo un'applicazione di tipo diverso avrò un rendimento migliore rispetto il progetto attuale.

In conclusione ho capito che devo migliorare nella pianificazione di progetti Agili e di migliorare la collaborazione nei lavori di gruppo.



10 Glossario

Termine	Descrizione
Agile	Metodologia di sviluppo software basata su iterazioni brevi e miglioramenti continui del prodotto.
C#	Linguaggio di programmazione sviluppato da Microsoft
CRUD	Create, Read, Update, Delete: Indica le operazioni principali eseguibili sui dati
IDE	Integrated Development Environment: Ambiente di sviluppo che fornisce strumenti per scrivere, eseguire e testare il codice.
.NET MAUI	Multi-platform App UI: Framework Microsoft per lo sviluppo di applicazioni multipiattaforma con un'unica base di codice.
NuGet	Gestore di pacchetti utilizzato per installare librerie e componenti nei progetti .NET
SQLite	Database relazionale leggero utilizzato per salvare i dati localmente sul dispositivo.
SQLite-net-pcl	Libreria .NET che facilita l'accesso e la gestione del database SQLite tramite codice C#.
Sprint	Periodo di lavoro definito all'interno della metodologia Agile durante il quale vengono sviluppate specifiche funzionalità.
XAML	eXtensible Application Markup Language: Linguaggio utilizzato in .NET MAUI per definire l'interfaccia grafica dell'applicazione.
API	Application Programming Interface: Insieme di funzioni che permettono la comunicazione tra applicazioni o componenti software.
DDR4	Double Data Rate 4: Tecnologia utilizzata per la memoria RAM.
GUI	Graphical User Interface: Interfaccia grafica con cui interagisce l'utente.
HDD	Hard Disk Drive: Dispositivo di archiviazione dati.
JSON	JavaScript Object Notation: Formato utilizzato per lo scambio di dati strutturati.
MySQL	My Structured Query Language: Sistema di gestione di database relazionali.
PCL	Portable Class Library: Tecnologia per condividere codice tra più piattaforme .NET.
RAM	Random Access Memory: Memoria utilizzata temporaneamente durante l'esecuzione dei programmi.
REQ	Requirement: Identificatore utilizzato per i requisiti del progetto.
SDK	Software Development Kit: Insieme di strumenti necessari per sviluppare applicazioni.
UI	User Interface: Interfaccia utente.
XAML	eXtensible Application Markup Language: Linguaggio usato per creare interfacce grafiche in .NET MAUI.
XAMPP	Pacchetto software che include Apache, MySQL e altri strumenti per lo sviluppo locale.



11 Bibliografia

- <https://www.drawio.com/> - Draw Io, 24.04.2026 – Utilizzato per la progettazione
- <https://moqups.com/it/> - Moqups, 17.04.2026 – Utilizzato per la realizzazione dei Mockup
- <https://moodle.edu.ti.ch/> - Moodle, 30.01.2026 – 29.05.2026 – Utilizzato durante il corso del Progetto e il modulo dell'anno scorso (318)
- <https://dotnet.microsoft.com/it-it/apps/maui> - DotNet Microsoft, 30.01.2026 – Ricerca dei calendari
- <https://chatgpt.com/> - ChatGPT, 30.01.2026 – 29.05.2026 – Utilizzato per risolvere problematiche nel codice
- <https://claude.ai/new> - Claude, 30.01.2026 – 29.05.2026 – Utilizzato per risolvere problematiche nel codice
- <https://teams.cloud.microsoft/> - Teams, 30.01.2026 – 29.05.2026 – Chat di gruppo per l'inoltamento dei file e varie comunicazioni
- [first app tutorial](#) – Microsoft – 27.02.2026 – Sito ufficiale di Maui per impostare l'app
- [notification](#) – Microsoft – 27.03.2026 – 17.04.2026 – Sito ufficiale di Maui per l'implementazione delle notifiche
- [database SQLite](#) – Microsoft – 13.03.2026 – Sito ufficiale di Maui per l'implementazione delle notifiche
- <https://www.google.com/imghp> – Google Immagini – Per cercare le immagini necessarie
- <https://ch.pinterest.com/> – Pinterest – Per cercare le immagini necessarie
- <https://stackoverflow.com/> – StackOverflow – 17.04.2026 – Trovare soluzioni a determinanti problemi durante la creazione del calendario
- <https://github.com/> – GitHub – 30.01.2026 – 29.05.2026 – Usato per condividere e collaborare alla scrittura di codice
- <https://outlook.cloud.microsoft.com> – Outlook – 30.01.2026 – 29.05.2026 – Servizio mail usato per inviarci i file

12 Allegati

- [Diagramma ER](#)
- [Gantt Preventivo](#)
- [Gantt Consuntivo](#)
- [Swimline](#)
- [Diagramma di flusso](#)
- [UseCase](#)
- [Manuale](#)
- [Diario di lavoro](#)
- [QdC](#)